



INTRODUCTION TO CONVOLUTIONAL NEURAL NETWORK

INT301 Bio-computation, Week 5, 2025

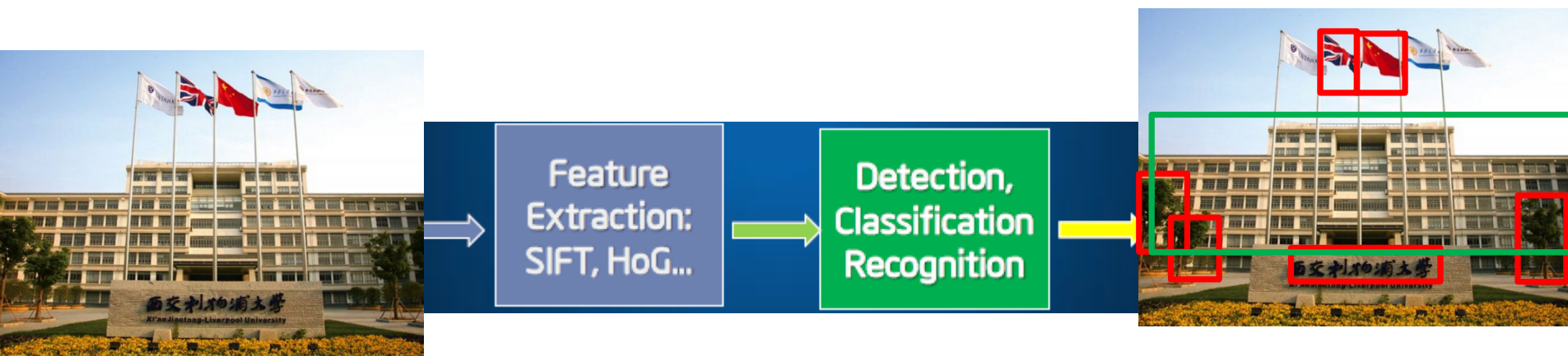


传统的计算机视觉方法

Classical Computer Vision Pipeline

我们需要一样样地去定义现有的特征，然后将现有的特征喂给机器学习模型

1. Select / develop features (e.g., HoG, SIFT, ...)
2. Add machine learning on top of this for recognition and train classifier



缺点：领域特定，需要人工设计，而且非常耗时。

☆ **Classical CV feature definition is domain-specific, hand engineered, and time-consuming**

Features for machine learning

image

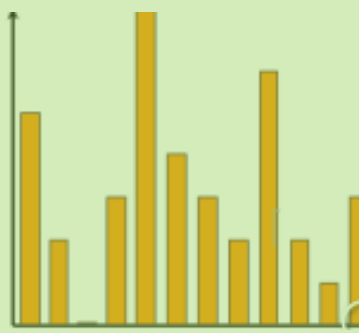
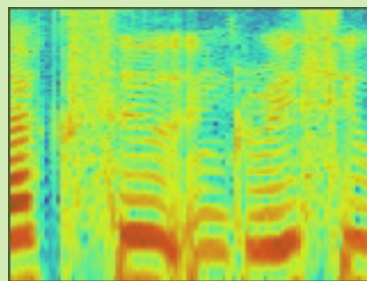
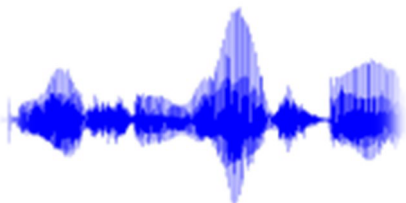


Image feature



Object detection
/classification

audio



Audio feature



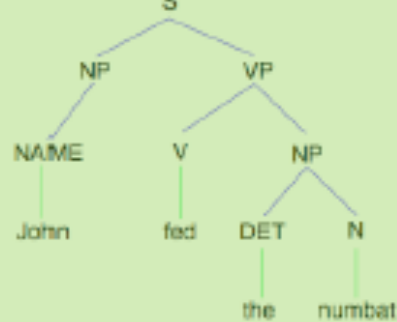
Speaker ID

text

央视网消息（新闻联播）：欢迎仪式后，习近平主席同高克总统在人民大会堂举行会谈，两国元首同意，继续巩固和加强中德全方位战略伙伴关系，在应对国际挑战中更加紧密合作。

习近平欢迎高克首次对 中国 进行国事访问。习近平指出，当前中德交流合作紧密程度超过历史任何时期。在双方共同努力下，中德关系发展水平正在迈向更高层次，政治互信持续提升，务实合作不断加强，人文交流更加广泛，在国际和地区问题上沟通协调更加频繁。

习近平强调，巩固和加强中德全方位战略伙伴关系，最重要的就是从战略高度和长远角度出发，牢牢把握两国关系发展方向。双方要坚持相互尊重、平等相待，照顾和尊重彼此核心利益和重大关切，努力扩大共同点，缩小分歧面，深化政治互信。要密切高层交往，用好现有的磋商对话机制。中方赞赏 德国 始终坚持一个中国政策，希望德方继续秉持这一积极立场。双方正在加紧落实《中德合作行动纲要》提出的合作共识和倡议，下一步可以把三方合作作为切入点，共同支持和参与“一带一路”和亚欧互联互通建设，开拓国际市场。两国应该积极加强在气候变化、安全等国际事务中的合作，加强在 联合国 等多边框架内的协调和配合。中德将分别主办今年明年二十国集团领导人会议，中方愿同德方就二十国集团重要议程保持密切沟通。双方要在文化、教育、媒体、青年、体育等广泛领域架设更多交流桥梁，办好中德青少年交流年活动。



Text feature

Translation/
/categorization
/Web search

Key Ideas of Deep Learning

- Deal with non-linear system
- Learn feature from data (or big data)
- Build feature hierarchies (function composition)
- End-to-end learning

Learning Feature Representations

The idea:

- Most perception (input processing) in the brain may be due to **one learning algorithm**.
- Build learning algorithms that mimic the brain.
- Most of human intelligence may be due to **one learning algorithm**.



one learning algorithm → end-to-end system!

卷积层的工作方式

Learning Feature Hierarchy

滤波器(filter) 和卷积核(kernel) 全连接层(fully connected layer).

Deep Learning

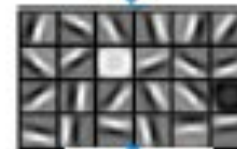
- Deep architectures can be representationally efficient.
- Natural progression from low level to high level structures.
- Can share the lower-level representations for multiple tasks.



3rd layer
“Objects”



2nd layer
“Object parts”



1st layer
“edges”



Input

1. 卷积层 (Convolutional Layer) – “细节侦察员” 在每个卷积层中，都有一系列可学习的滤波器，提取图像的局部特征。

任务: 识别局部特征。

工作流程:

Filter

CNN 的第一层卷积层直接作用于输入的原始图像。它使用多个滤波器（卷积核）来扫描图像。每个滤波器都经过训练，专门用来寻找一种非常基础的视觉模式，例如：水平边缘、垂直边缘、特定颜色的斑块、角点等。当一个滤波器在图像的某个区域上找到了它要找的模式时，它就会在输出的特征图 (Feature Map) 上的相应位置产生一个高激活值。因此，经过第一层卷积层后，我们得到了一组特征图，每一张图都描绘了原始图像中某一种基础特征的分布情况。它保留了特征的空间位置信息。

filter 的数量 = Feature maps 的数量；
一个 filter 结果对应一个 feature map.

2. 池化层 (Pooling Layer) – “区域总结员”

任务: 降低特征图的复杂度，并使其对位置变化不那么敏感。

工作流程: 池化层紧跟在卷积层之后，对卷积层输出的特征图进行处理。通过降采样 (Down-sampling) 的方式，将特征图的尺寸（高度和宽度）缩小。最常用的方法是最大池化 (Max Pooling)，它只保留每个小区域内最强的特征信号。

这样做的好处是：减少了数据量，使得后续的计算更加高效；提取了最显著的特征，忽略了不重要的细节；提供了平移不变性，即使图像中的物体有轻微的移动，池化后的结果依然稳定。

阶段二：分类决策 (Classification)

这个阶段的目标是利用前面提取到的高级特征，最终判断这张图片属于哪个类别。

3. 全连接层 (Fully Connected Layer) – “首席决策官”

任务: 根据提取到的特征进行综合判断和分类。

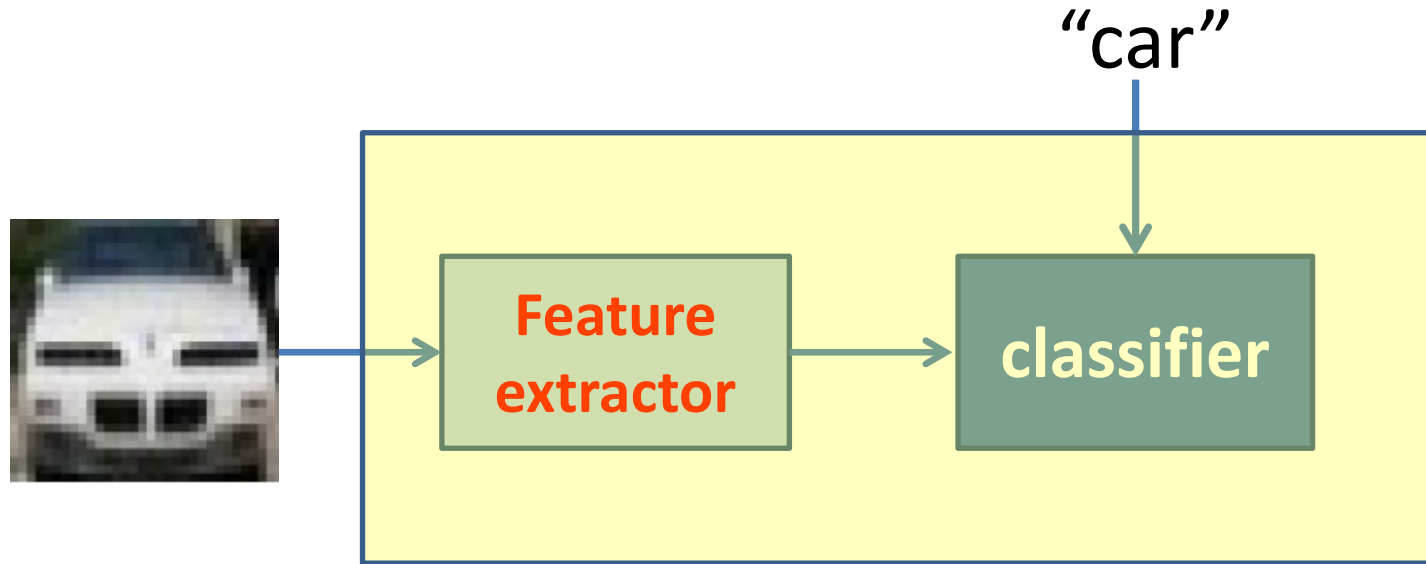
工作流程: 展平 (Flatten): 首先，我们需要将特征提取阶段输出的最后一个三维特征图（例如 $5 \times 5 \times 64$ ）“压扁”，变成一个长长的一维向量。这个向量可以看作是整张图片所有高级特征的“总结报告”。加权求和: 这个一维向量被输入到全连接层。全连接层的工作方式与传统的神经网络完全相同：每个输入节点都连接到下一层的每个节点。它通过学习权重，来判断哪些高级特征的组合对于区分不同的类别最重要。

例如，如果“猫耳”特征和“胡须”特征同时被激活，那么这张图片是“猫”的权重就会非常高。

最终输出: 网络的最后一层是一个特殊的全连接层（通常带有 Softmax 激活函数），它的神经元数量等于我们要分类的类别总数。它会输出一个概率列表，例如：[猫的概率: 95%，狗的概率: 4%，鸟的概率: 1%]。我们选择概率最高的那个作为最终的识别结果。

每个卷积层中都有一系列可以学习的滤波器，用于提取图像的基本特征。每个卷积层中包含的滤波器可能都不相同，因为需要负责不同层次的提取任务。

End-to-end Object Recognition



如何利用数据优化特定任务的功能

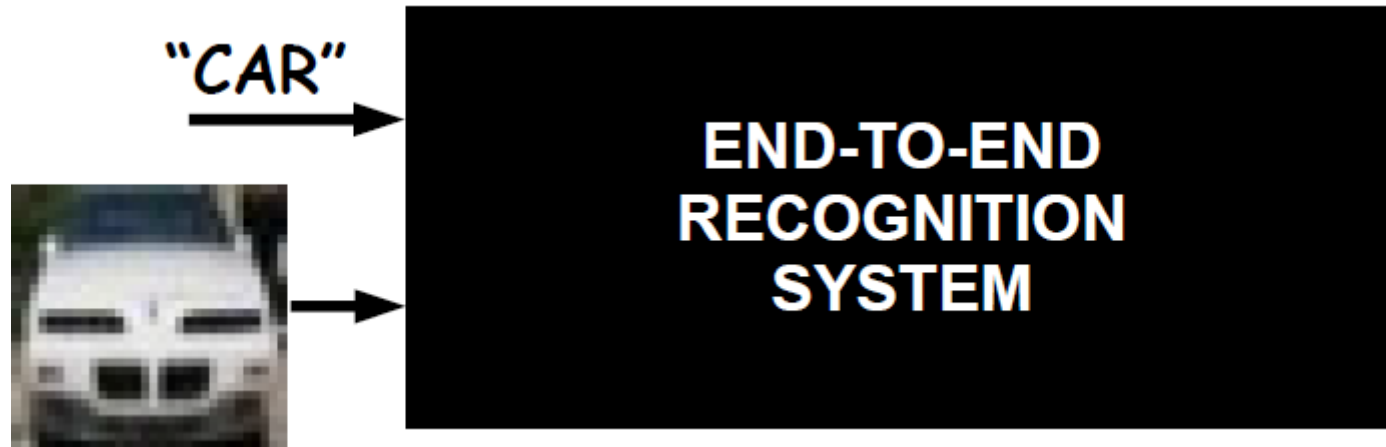
How to use data to **optimize features** for the given task?

- Everything becomes adaptive. 一切都变得自适应
- No distinction between feature extractor and classifier.
- Big non-linear system trained from raw pixels to labels

基于原始像素到标签训练的大型非线性系统

特征提取器和
分类器不做区分

End-to-end Object Recognition



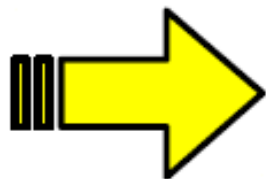
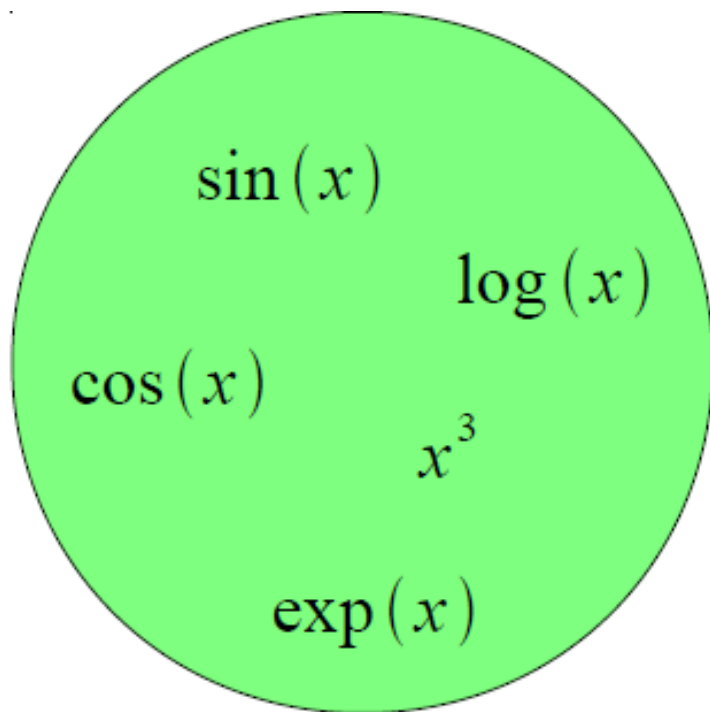
Q: How can we build such a highly non-linear system?

通过组合简单的构建模块，我们可以创建出越来越复杂的系统。

A: By combining simple building blocks, we can make more and more complex systems.

Building A Complicated Function

Simple Functions



One Example of Complicated Function

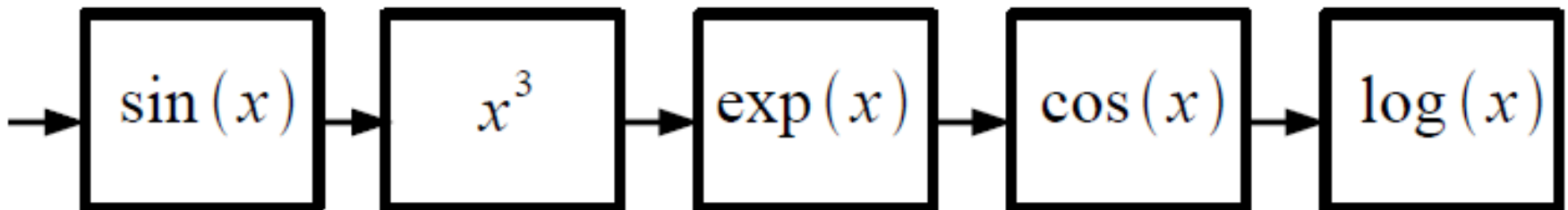
A red oval containing the complicated function $\log(\cos(\exp(\sin^3(x))))$.

- Function composition is at the core of deep learning methods.
每个“简单函数”都将包含需要训练的参数。
- Each “simple function” will have parameters subject to training.

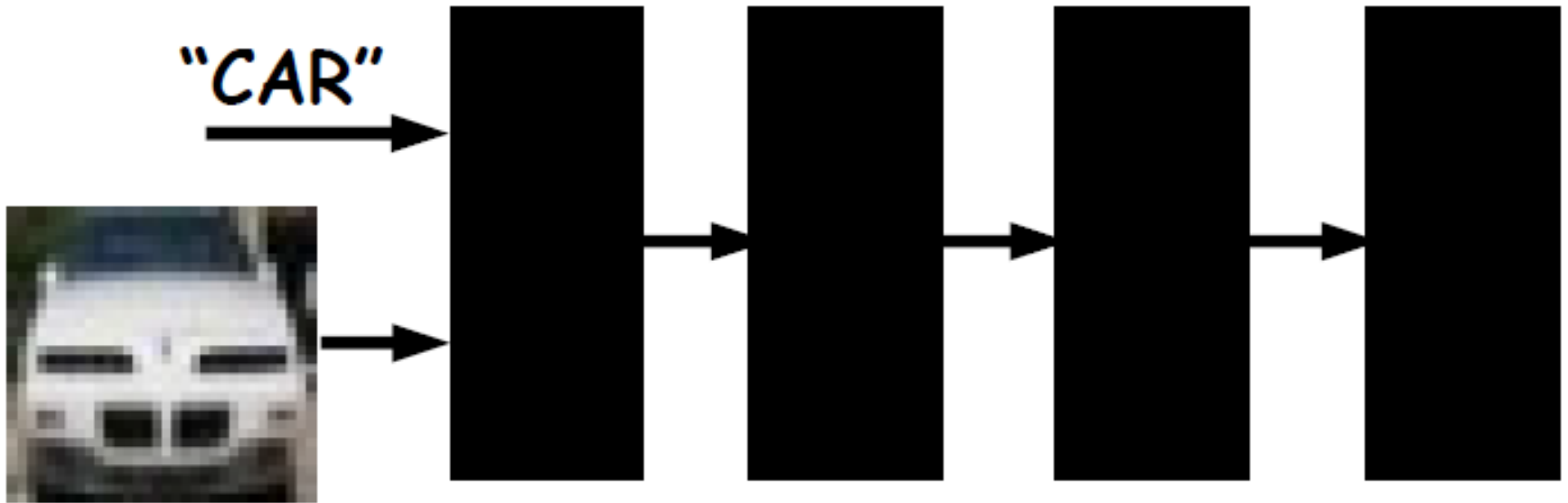
Building A Complicated Function

Complicated Function

$$\log(\cos(\exp(\sin^3(x))))$$



Intuition Behind Deep Neural Nets

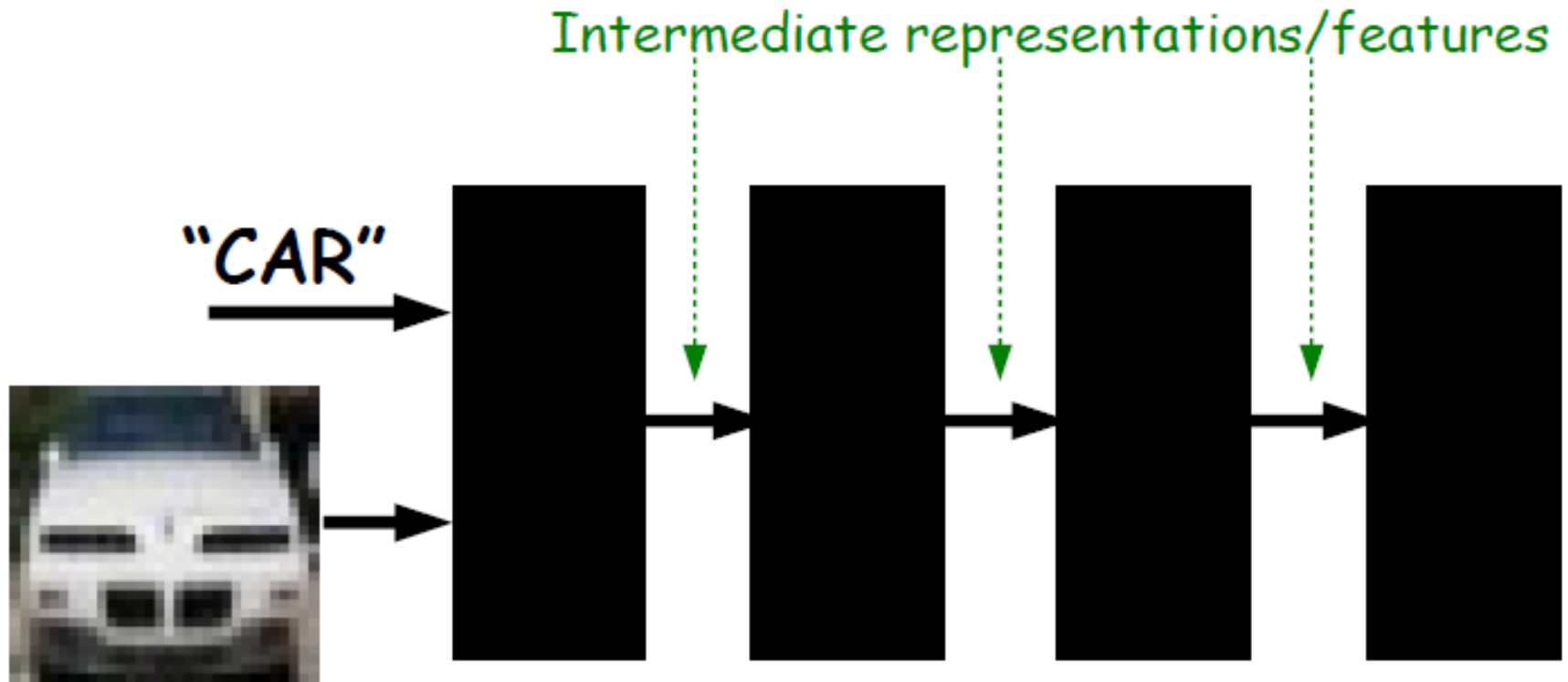


NOTE:

Each black box can have trainable parameters.

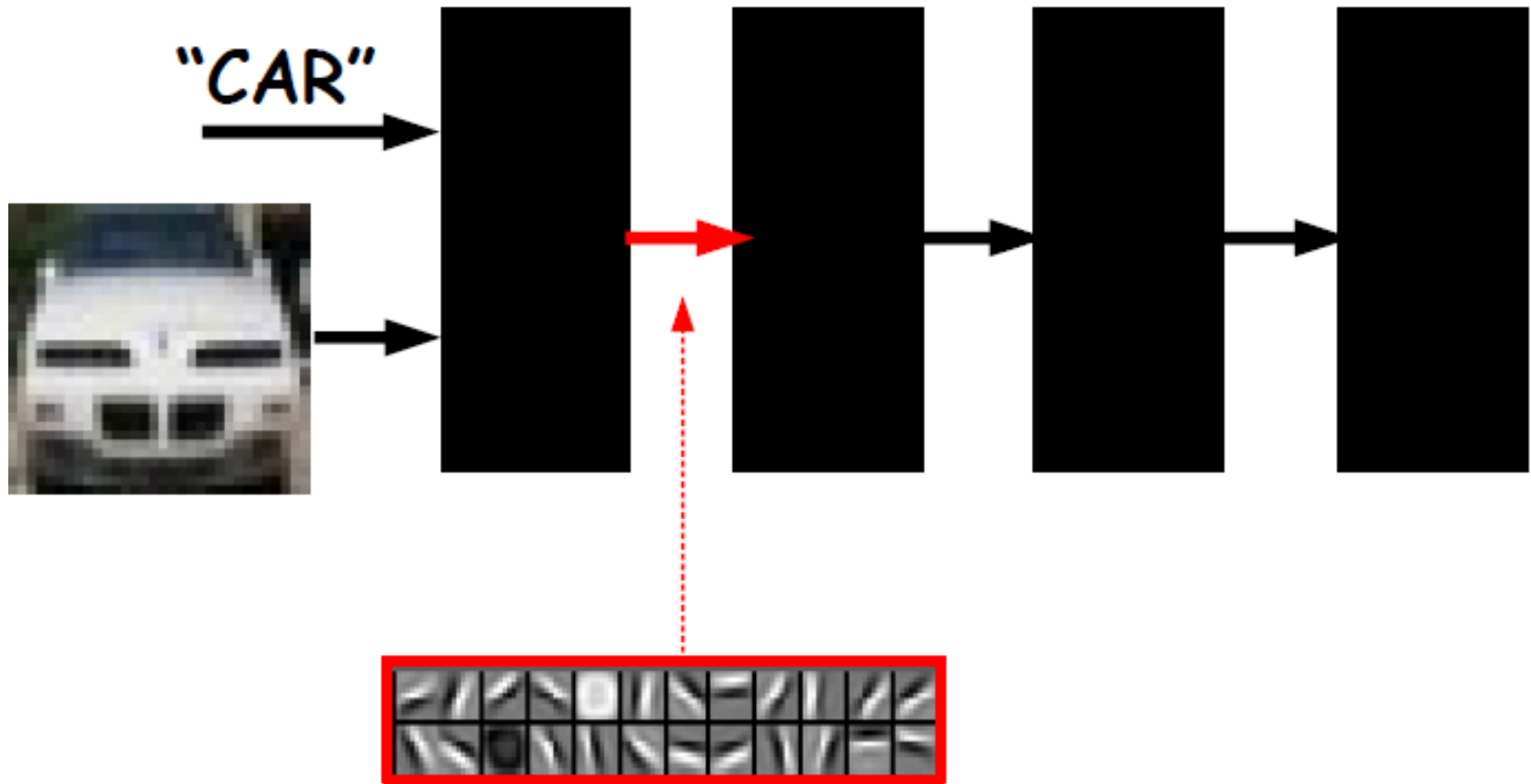
Their composition makes a highly non-linear system.

Intuition Behind Deep Neural Nets

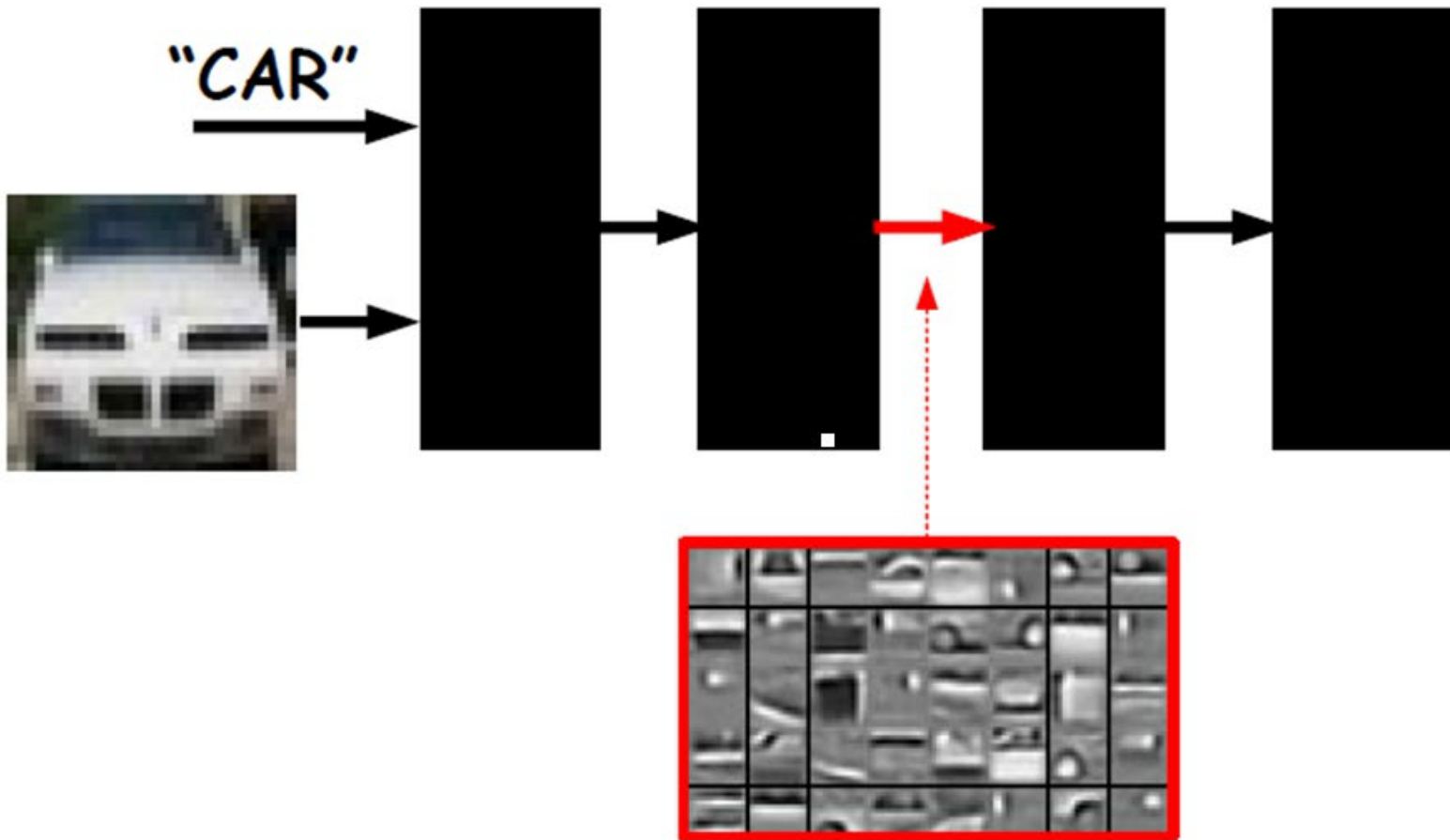


NOTE: System produces a **hierarchy** of features

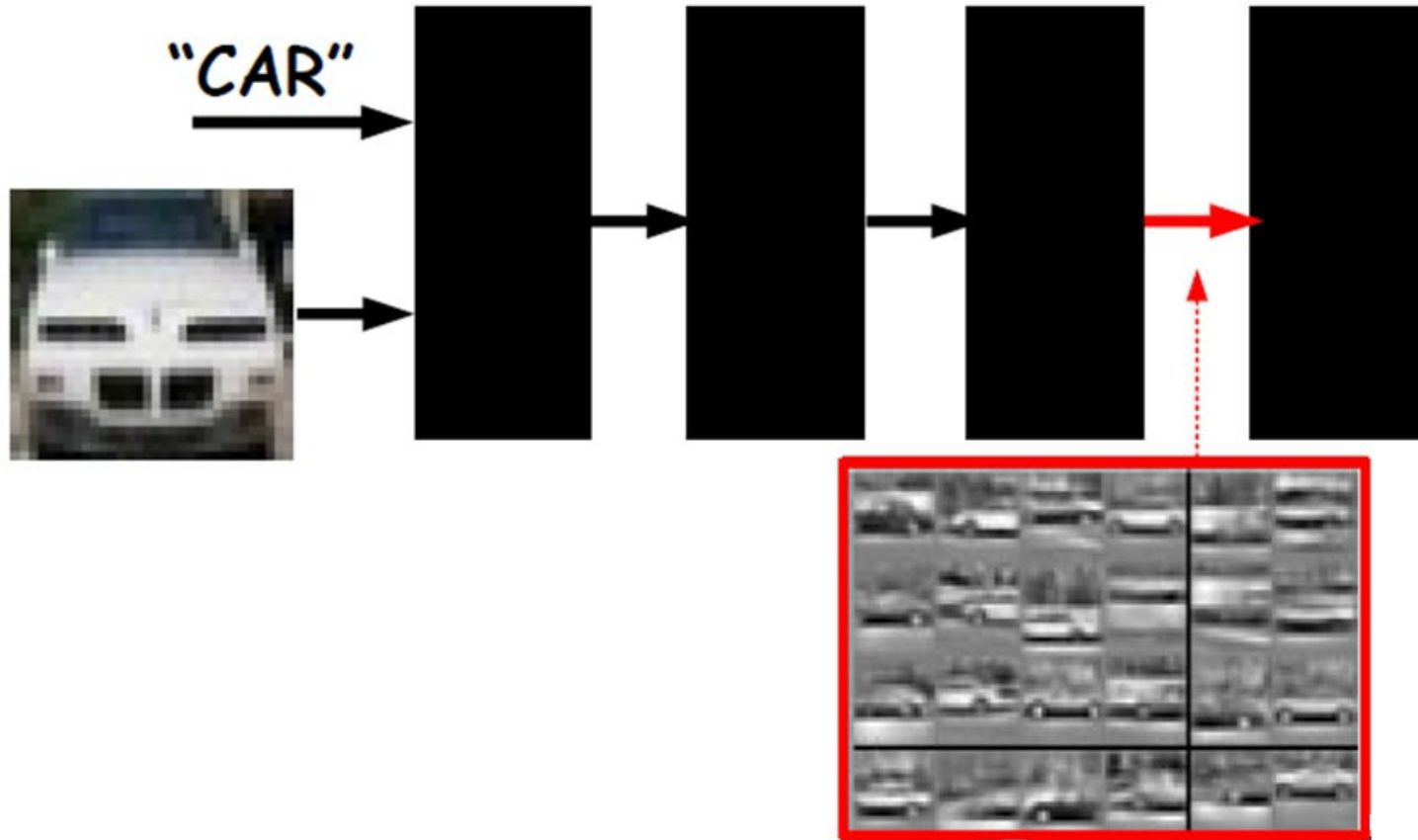
Intuition Behind Deep Neural Nets



Intuition Behind Deep Neural Nets



Intuition Behind Deep Neural Nets



输入→特征提取（卷积层+池化层）→转换→分类决策（全连接层）→输出

我们把你提供的这几张图按顺序串起来看，正好就是我刚才描述的“层次化的特征学习” (Hierarchical Feature Learning)过程：

第一阶段 (对应幻灯片13): 这是紧跟在输入图像后的第一个“黑盒子”（第一个卷积层）。它学习到的特征，都是非常基础的边缘、线条和角点。这正是我们说的，浅层网络负责识别最简单的局部模式。

第二阶段 (对应幻灯片14): 这是中间的“黑盒子”（某个中间卷积层）。它不再直接看原始图片，而是看第一阶段输出的“边缘和线条图”。然后，它把这些简单的线条组合起来，形成了更复杂的物体部件 (Object parts)，比如车轮的弧度、车灯的轮廓、格栅的形状。

第三阶段 (对应幻灯片15): 这是更深层的“黑盒子”。它又把第二阶段的“物体部件”进行组合，最终学习到了能够代表整个物体的、更加抽象和完整的特征，比如不同角度下的车头。

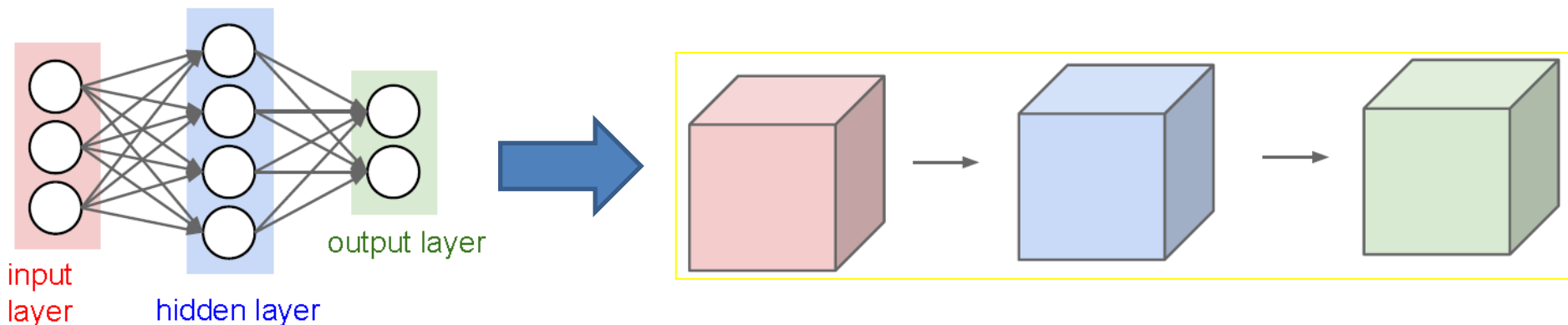
层次化的特征学习 (Hierarchical Feature Learning)

CNN 的强大之处在于，卷积层 -> 池化层 这个组合可以被堆叠多次。

1. 浅层网络 (靠近输入): 卷积层学习到的是非常简单、基础的特征（边缘、颜色、纹理）。
2. 中层网络: 后续的卷积层会以前一层输出的特征图作为输入。它们不再看原始像素，而是看由基础特征组成的模式。因此，它们能学习到更复杂的特征，比如眼睛的轮廓、鼻子的形状（通过组合边缘和弧线得到）。
3. 深层网络 (靠近输出): 更深层的卷积层则能将中层特征组合起来，识别出物体的特定部分，甚至是整个物体（例如，通过组合眼睛、鼻子、嘴巴的特征来识别人脸）。

到此为止，特征提取阶段结束。我们得到的是一组高度抽象化的、尺寸较小的特征图，它们包含了识别物体所需的关键信息。

Convolutional NN



局部感受野 (Local receptive fields):

对应概念: 这就是卷积层中滤波器 (Filter) / 卷积核 (Kernel) 的工作方式。

解释: 在传统的神经网络里 (图左侧), 输入层的每个神经元都要和隐藏层的所有神经元连接。如果输入是一张大图片, 那连接数简直是天文数字! 而 CNN 的卷积层不同, 每个神经元只关注输入图像中的一小块局部区域 (比如 3×3 或 5×5), 这就是它的“局部感受野”。它只处理自己“视野”内的信息, 就像拼图一样, 每个人只负责一小块, 最后再整合。

- Convolutional Neural Networks is extension of traditional Multi-layer Perceptron, based on 3 ideas:

1. Local receive fields

2. Shared weights

3. Spatial / temporal sub-sampling

权值共享 (Shared weights):

对应概念: 这也是卷积层的核心思想。

解释: 这个概念和上面是紧密相连的。一个滤波器 (比如“竖线探测器”) 在扫描整个图片时, 它自身的参数 (权重) 是不变的。也就是说, 我们用同一个“探测器”去检查图片的所有位置。这就是“权值共享”。这样做的好处是巨大的:

1. 参数量大大减少: 我们不需要为图片上每个位置都单独训练一个探测器。
2. 平移不变性: 无论“竖线”出现在图片的左上角还是右下角, 都能被同一个滤波器识别出来。

空间/时间子采样 (Spatial / temporal sub-sampling):

对应概念: 这指的就是池化层 (Pooling Layer)。

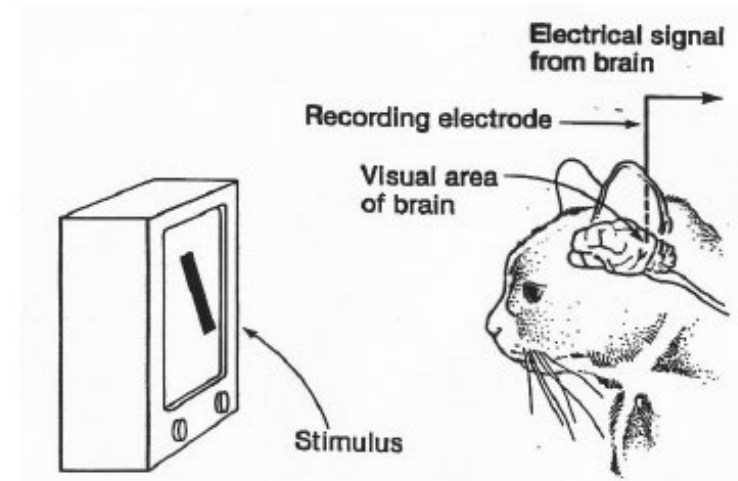
解释: “空间子采样”这个词听起来很专业, 但它描述的正是我们说的池化操作。通过选取一个区域内的最大值 (Max Pooling) 或平均值, 我们对特征图进行了采样, 减小了它的空间尺寸 (分辨率), 从而达到了降维和提炼特征的目的。

A bit of history

Hubel & Wiesel

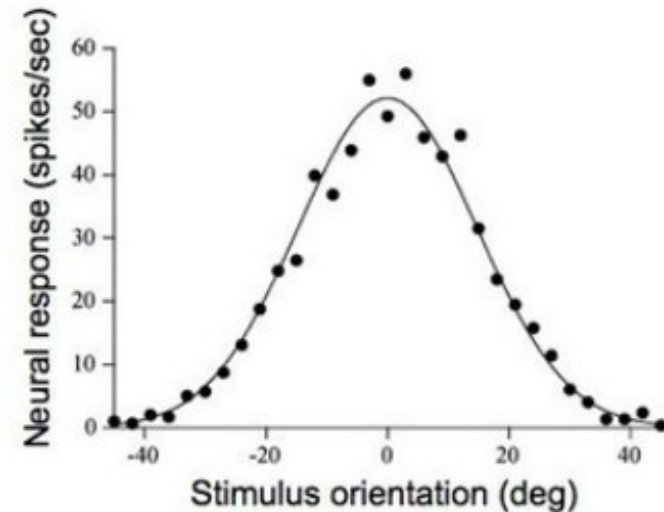
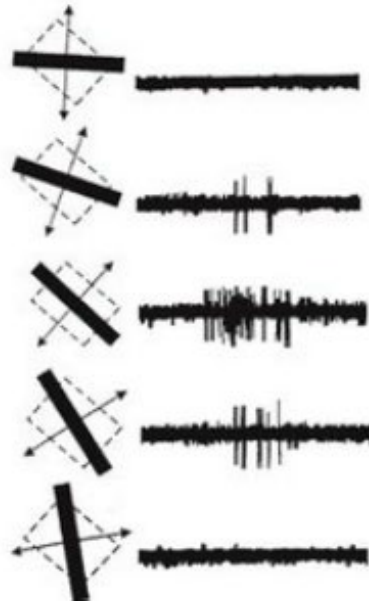
1959 Paper

Receptive fields of single neurones in the cat's striate cortex



1962 Paper

Receptive fields, binocular interaction and functional architecture in cat's visual cortex

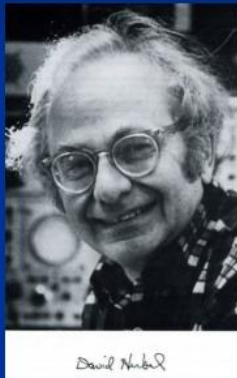


A bit of history

Topographical mapping in the cortex:

nearby cells in cortex
represented nearby
regions in the visual field

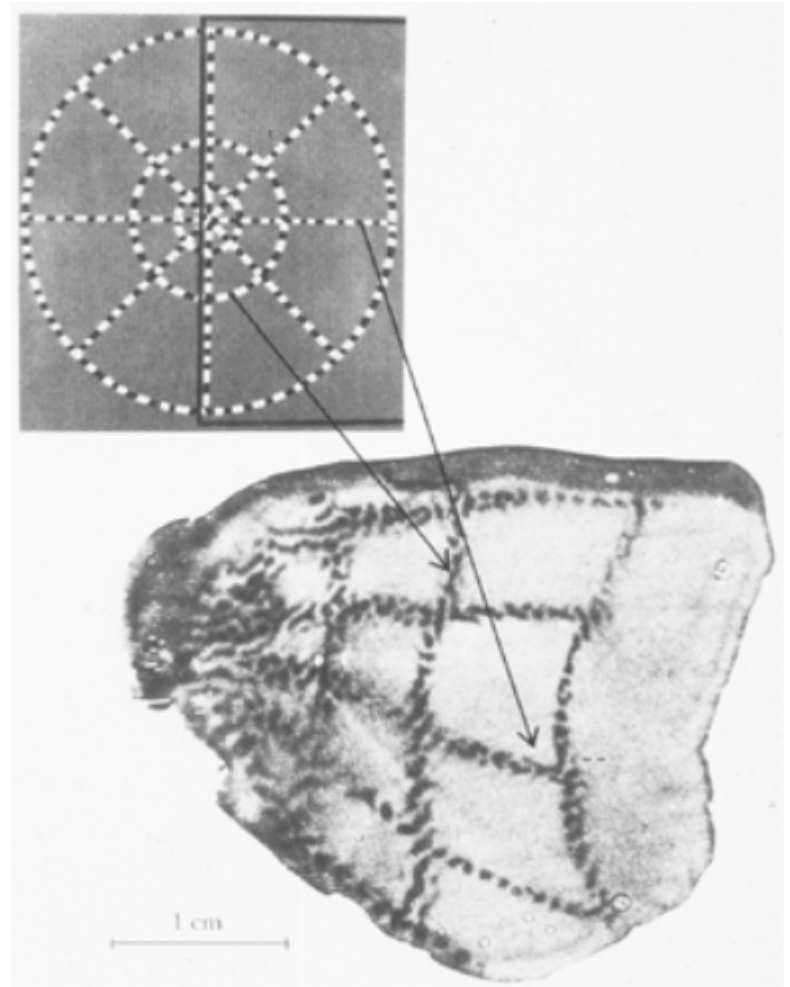
Nobel Prize 1981



David Hubel



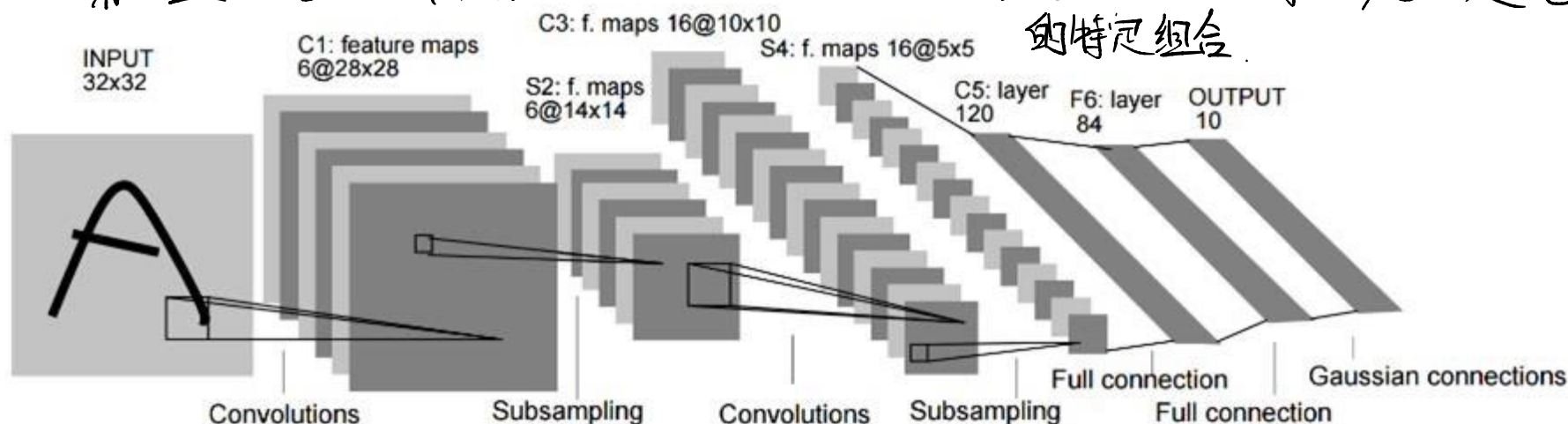
Torsten Wiesel



LeNet [LeCun et al. 1998]

第一组: $C1 \rightarrow S2$ 用来学习图像中的边缘、角点等底层逻辑

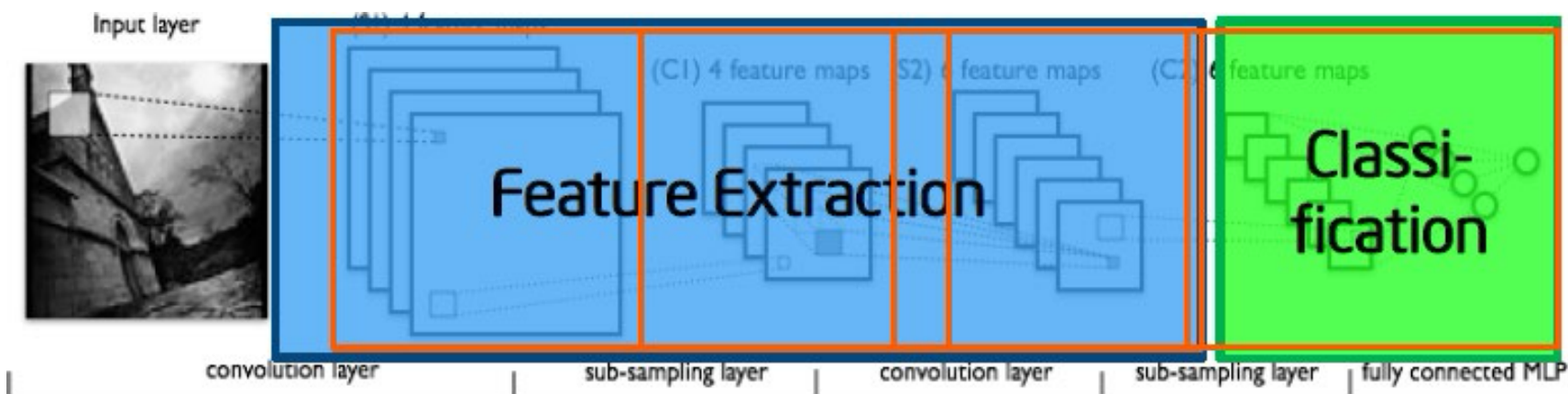
第二组: $C3 \rightarrow S4$ 在第一组提取的特征基础上, 学习复杂的组合特征, 比如数字笔画的特定组合



- See LeCun paper (1998) on text recognition:

<http://yann.lecun.com/exdb/publis/pdf/lecun-01a.pdf>

Convolutional NN (CNN)



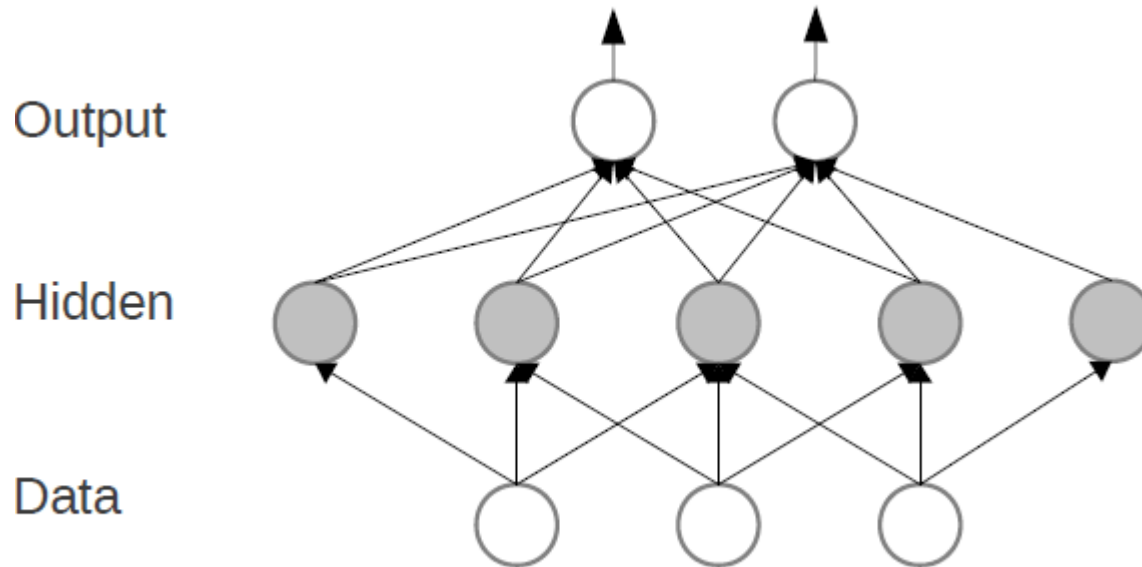
- Convolutional layer 卷积层
- Sub-sampling layer 池化层
- Fully connected layers 全连接层

Basic Concept of CNN

- Here's a one-dimensional convolutional neural network
- Each hidden neuron applies *the same localized, linear filter* to the input

Localized (局部化): 你看图中的 Hidden 层，每个神经元只连接到 Data 层的一小部分（2个输入），而不是全部。这就是“局部感受野”。

The same filter (同一个滤波器): 虽然图中没有明确画出参数，但它想表达的是，Hidden 层从左到右的每一个神经元，都使用完全相同的权重（也就是同一个滤波器）来处理它们各自看到的局部输入。这就是“权值共享”。



Sparse Connectivity

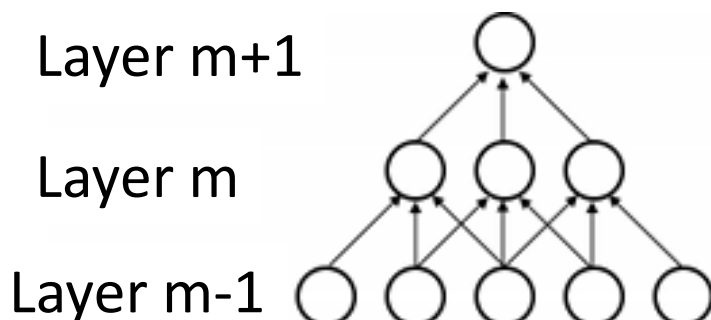
CNN中的神经元只与前一层空间上相邻的一小部分神经元连接，而不是像传统神经网络那样“全员连接”

Sparse connectivity

- CNNs exploit spatially-local correlation by enforcing a local connectivity pattern between neurons of adjacent layers, i.e., the inputs of hidden units in layer m are from a subset of units in layer $m-1$, units that have spatially contiguous *receptive fields* 感受野

空间局部相关性

在图像这类数据中，像素之间的关联性通常是局部的。一个像素和它旁边的像素关系很密切，但和离得很远的像素基本没什么关系。CNN的稀疏连接正是利用了这种“空间局部相关性”



对于一个神经元来说，它的感受野是指在输入空间（也就是最开始的原始图片）中，能够影响到这个神经元激活状态的区域。

图中 Layer m 里的一个神经元，它直接连接到 Layer $m-1$ 的3个神经元，所以它相对于 Layer $m-1$ 的感受野是3。虽然 Layer $m+1$ 的神经元相对于 Layer m 的感受野也是3，但它相对于最底层输入（the input retina）Layer $m-1$ 的感受野已经是5了。这是因为它可以通过 Layer m 直接看到所有的 Layer $m-1$ 。

这个特性是 CNN 能够学习层次化特征的结构基础。通过一层一层地堆叠，深层网络的神经元就能够“看”到越来越大的原始图像区域，从而能将前面学到的小特征（如边缘、角点）组合成更复杂的、更大范围的特征（如眼睛、鼻子）。

正是因为神经元只关注局部区域（稀疏连接），整个网络的架构就被“强制”去学习那些能够对空间局部输入模式产生最强响应的“滤波器”。每个滤波器本质上就是一个权重模版，它的任务就是在自己的感受野内去寻找特定的局部模式

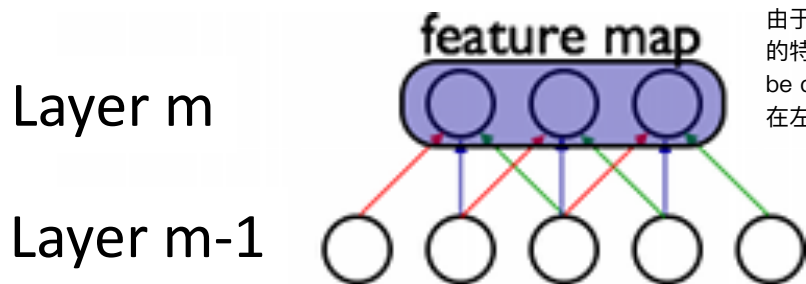
Imagine that layer $m-1$ is the input retina. In the above figure, units in layer m have receptive fields of width 3 in the input retina and are thus only connected to 3 adjacent neurons in the retina layer. Units in layer $m+1$ have a similar connectivity with the layer below. We say that their receptive field with respect to the layer below is also 3, but their receptive field with respect to the input is larger (5). Each unit is unresponsive to variations outside of its receptive field with respect to the retina. The architecture thus ensures that the learnt “filters” produce the strongest response to a spatially local input pattern.

Shared Weights

每个滤波器在整个视野中被复制 (replicated)”。你可以想象我们只精心设计了一个“特征探测器”，然后把这个探测器的无数个“复制品”派出去，让它们同时在图片的不同位置上进行探测工作。

在生成一张特征图 (feature map) 的过程中，所有神经元使用的都是同一个滤波器，即共享同一套权重 (weights) 和偏置 (bias)。图中的 Layer m，那三个紫色的神经元共同构成了一张特征图。它们分别连接到 Layer m-1 的不同局部区域，但它们各自的连接权重模式 (用红、绿、蓝三种颜色的线表示) 是完全相同的。

- In CNNs, each filter is replicated across the entire visual field. These replicated units share the same parameterization (weight vector and bias) and form a *feature map*



1. 位置无关性 / 平移不变性 (Positional Invariance / Equivariance)
由于网络在图片的所有位置都使用同一个特征探测器，因此一个特定的特征无论出现在图像的哪个位置，都能被检测出来 (“features to be detected regardless of their position”)。例如一个车无论出现在左上角还是右下角都能被这个滤波器探测出来

2. 极高的参数效率 (Parameter Efficiency)

这是最直观的好处。假设我们有一个 5×5 的滤波器，要处理一张 100×100 的图片。如果没有权值共享，每个输出神经元都需要自己的一套 $5 \times 5 = 25$ 个权重，那参数量将会是天文数字。但在权值共享机制下，无论图片多大，我们只需要学习这一套 $5 \times 5 = 25$ 个权重参数就够了！

In the above figure, we show 3 hidden units belonging to the same feature map. Weights of the same color are shared—constrained to be identical. Gradient descent can still be used to learn such shared parameters, with only a small change to the original algorithm.

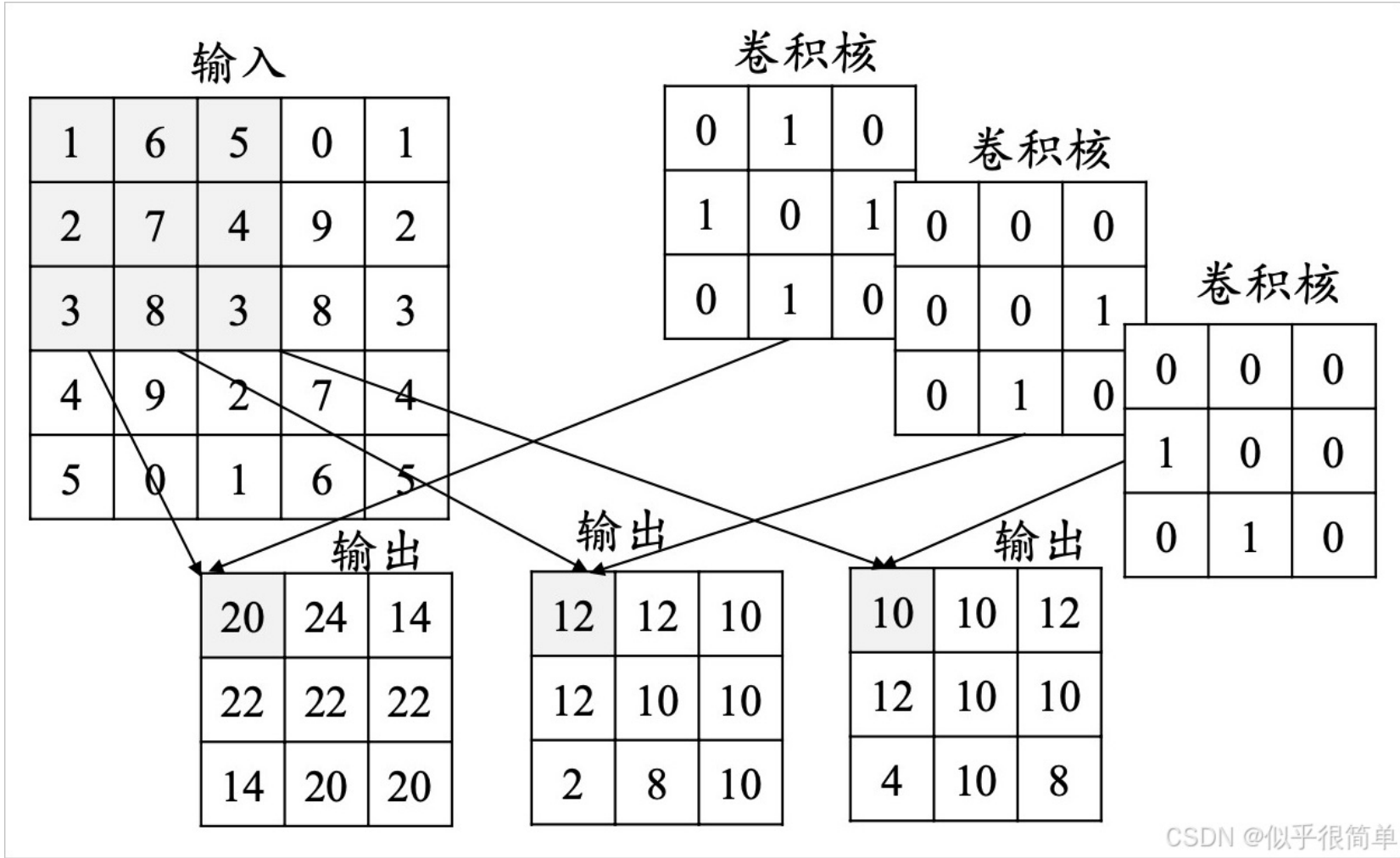
Replicating units in this way allows for features to be detected regardless of their position in the visual field. Additionally, weight sharing increases learning efficiency by greatly reducing the number of free parameters being learnt. The constraints on the model enable CNNs to achieve better generalization on vision problems.

卷积核 (Kernel/Filter)：即“特征提取器”，是一个小型的可学习矩阵（如 3×3、5×5），深度与输入特征图的通道数 (C) 一致。例：输入为 3 通道 (RGB)，卷积核的深度必为 3（每个通道对应一个 2D 核，最终结果求和）。

步长 (Stride)：卷积核在输入特征图上滑动的“步幅”，默认值为 1。步长越大，输出特征图尺寸越小。

填充 (Padding)：在输入特征图边缘补 0（或其他值），用于保持输出特征图尺寸与输入一致（“Same Padding”），或减少尺寸（“Valid Padding”）。

特征图 (Feature Map)：卷积操作的输出结果，每个卷积核对应一个特征图（通道数 = 卷积核数量）



生成 feature map 需要 3 步:

Details and Notation

1. 卷积 convolution: 用一个线性滤波器 (W^k , 权重) 与输入 X 进行卷积操作 ($*$)

- A feature map is obtained by repeated application of a function across sub-regions of the entire image, in other words, by *convolution* of the input image with a linear filter, adding a bias term and then applying a non-linear function. 2. 加偏置项 (adding a bias): 为结果加上一个偏置项 b_k 这是一个学习函数让滤波器能更灵活地调整
- If we denote the k -th feature map at a given layer as h^k , whose filters are determined by the weights W^k and bias b_k , then the feature map is obtained using convolution as follows (for *tanh* non-linearities): 非线性激活 (Applying a non-linear function)



$$h_{ij}^k = \tanh((W^k * X)_{ij} + b_k)$$

将结果通过一个非线性函数, 比如 $\tanh$ 或 ReLU 。这样赋予了网络学习复杂模式的能力。

Convolutional Layers

- Suppose that we have some $N \times N$ square neuron layer which is followed by our convolutional layer. If we use an $m \times m$ filter ω , our convolutional layer output will be of size $(N-m+1) \times (N-m+1)$. 一个 $N \times N$ 的输入, 经过一个 $m \times m$ 的滤波器卷积之后输出的特征图尺寸.
- In order to compute the pre-nonlinearity input to some unit x_{ij}^l in our layer, we need to sum up the contributions (weighted by the filter components) from the previous layer cells:

$$x_{ij}^l = \sum_{a=0}^{m-1} \sum_{b=0}^{m-1} \omega_{ab} y_{(i+a)(j+b)}^{\ell-1}$$

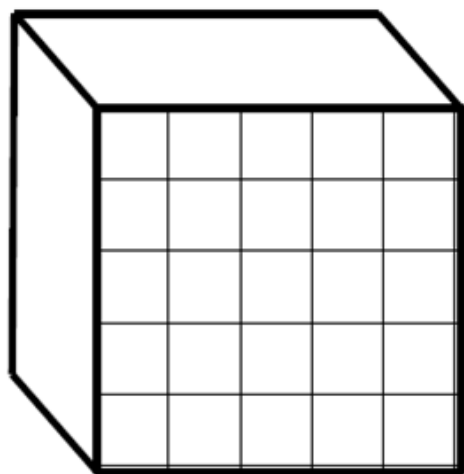
输出特征图上的 (i,j) 位置的值, 是通过将滤波器 ω 的权重与输入 y 上对应的小区域的像素值进行元素相乘后再求和

因为filter是3x3,
所以每一层都只能
看9个

Convolution in 2D

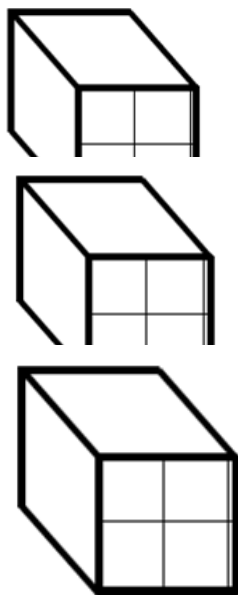
滤波器深度和输入
深度相同

Input "image"

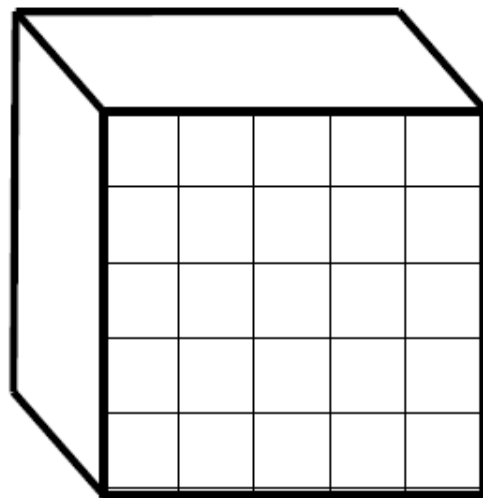


5x5

Filter bank



Output map



Convolutional layers :

a rectangular grid of neurons.

The previous layer is also a rectangular grid of neurons. 在一个卷积层中所有神经元的矩阵区域权重均相同。

Each neuron takes inputs from a rectangular section of the previous layer; the weights for this rectangular section are the same for each neuron in the convolutional layer

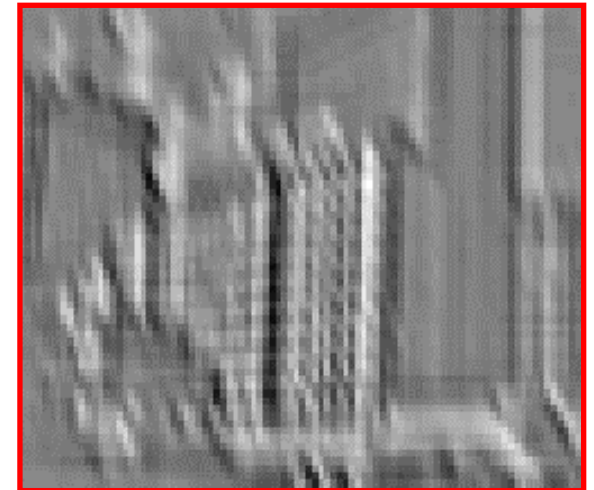
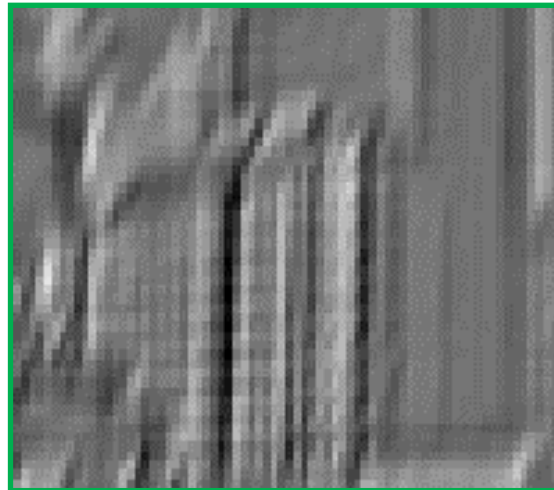
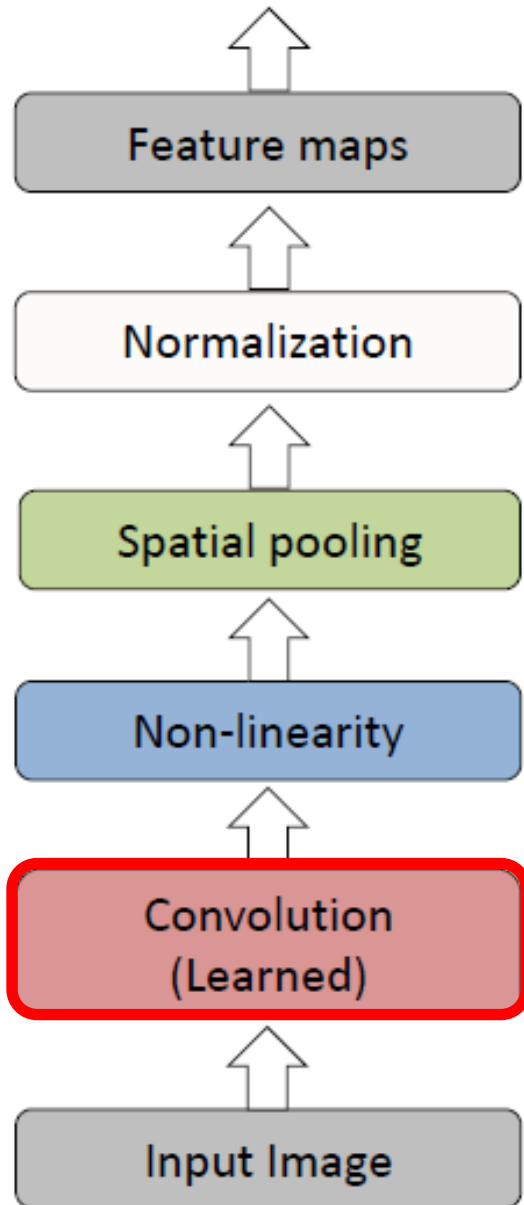
1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

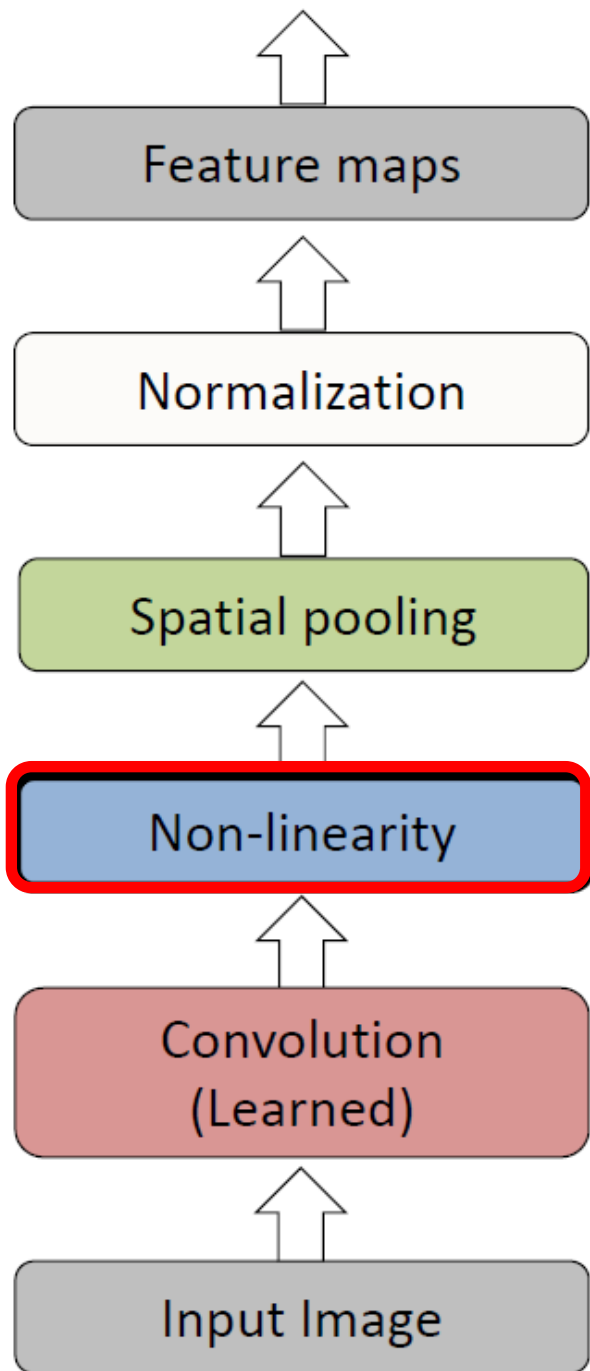
Image

4		

Convolved Feature

Convolutional Neural Networks



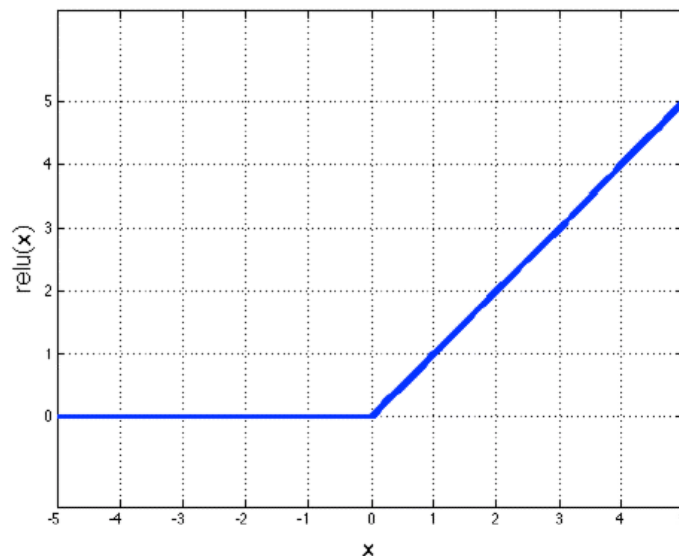


Non-saturating function

激活函数

$$f(x) = \max(0, x)$$

Rectified Linear Unit (ReLU)



很大程度上解决了深度网络训练中的梯度消失问题

计算速度快 (Computationally simpler): ReLU的前向和后向传播都只是一个简单的 if 判断 (if $x > 0$ then $y=x$ else $y=0$)。而像 sigmoid 这样的函数需要计算复杂的指数 e^x , 这在拥有数百万神经元的大型网络中会慢得多。

Benefits of using ReLU

- ReLUs are much simpler computationally
 - The forward and backward passes through an ReLU are both just a simple *if* statement
 - The sigmoid activation requires computing an exponent
 - This advantage is huge when dealing with big networks with many neurons, and can significantly reduce both training and evaluation times

缓解梯度消失问题 (Avoiding saturation):这是更重要的一个优势。Sigmoid 函数在输入值很大或很小时, 曲线会变得非常平坦 (饱和区)。在这些区域, 它的导数 (梯度) 几乎为0。在深度网络中, 反向传播需要将梯度连乘, 很多个接近0的数乘在一起, 梯度就会很快“消失”, 导致网络前端的层无法有效学习。

而 ReLU 对于所有正数输入, 梯度恒为1, 这使得梯度可以很顺畅地在网络中传播, 从而让训练非常深的网络成为可能。

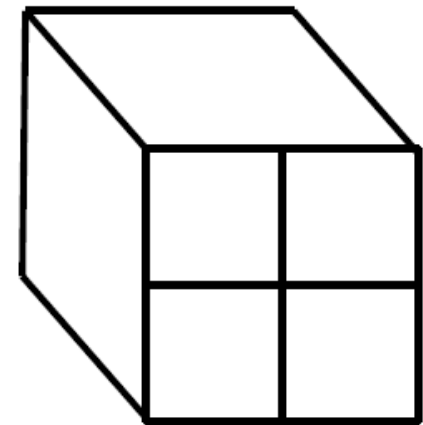
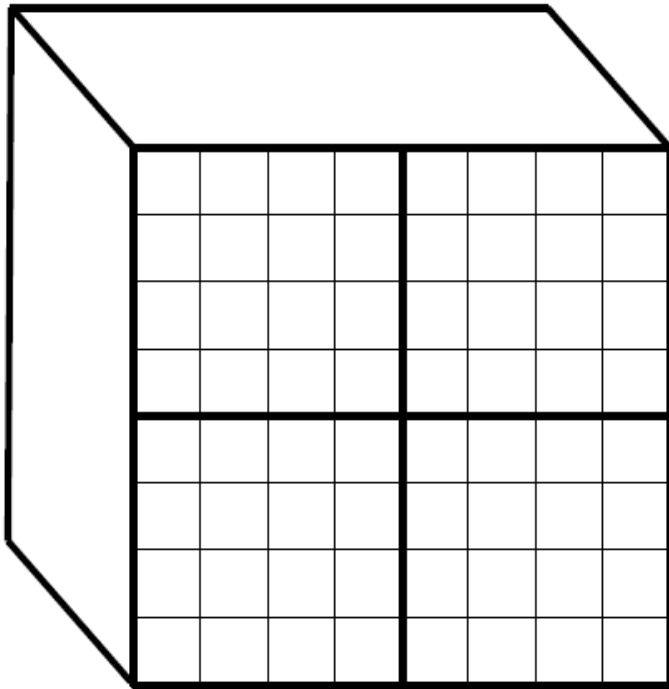
Benefits of using ReLU

- Sigmoid activations are easier to saturate
 - There is a comparatively narrow interval of inputs for which the sigmoid's derivative is *sufficiently nonzero*
 - In other words, once a sigmoid reaches either the left or right plateau, it is almost meaningless to make a backward pass through it, since the derivative is very close to 0
- ReLUs only saturate when the input is less than 0
 - Even this saturation can be eliminated using leaky ReLUs
- For very deep networks, saturation hampers learning, and so ReLUs provide a nice workaround

Local pooling operation

In order to reduce variance, pooling layers compute the max or average value of a particular feature over a region of the image. This will ensure that the same result will be obtained, even when image features have small translations

池化层通过“计算一个区域内某个特征的最大值或平均值 (compute the max or average value of a particular feature over a region)”来工作。从而“减少方差 (reduce variance)”，使特征表示更加精炼和紧凑。



池化操作“将确保即使图像特征有微小的平移，也能获得相同的结果 (will ensure that the same result will be obtained, even when image features have small translations)”。

More on Pooling operation

Subsampling (pooling) Mechanism

- The exact positions of the extracted features are not important
- Only relative position of a feature to another feature is relevant
- Reduce spatial resolution – Reduce sensitivity to shift and distortion

它体现了CNN的一个核心设计哲学：在物体识别中，“提取出的特征的精确位置并不重要 (The exact positions of the extracted features are not important)”。

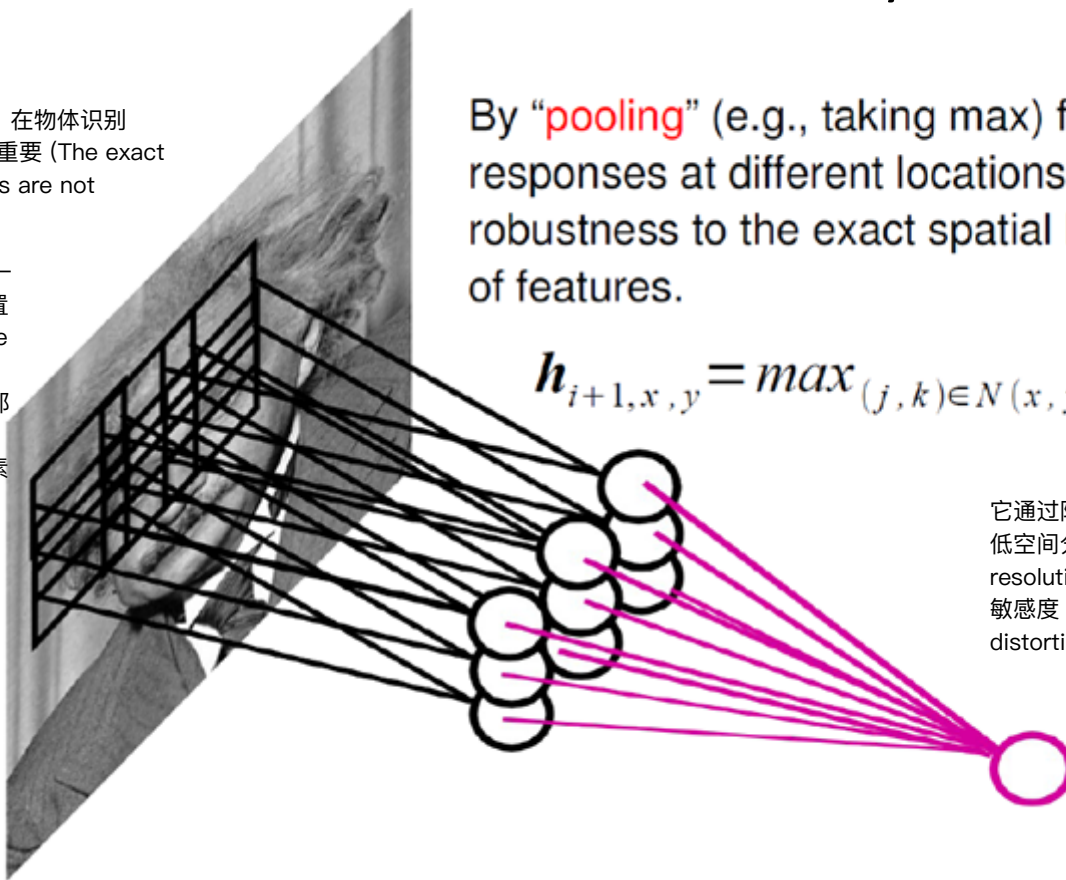
它关注的是相对关系：更重要的是“一个特征相对于另一个特征的相对位置 (Only relative position of a feature to another feature is relevant)”。

例如，对于字母“A”，重要的是“顶部交叉点”特征在两个“斜线”特征的上方，而不是它们各自在图片中的像素坐标。

By “pooling” (e.g., taking max) filter responses at different locations we gain robustness to the exact spatial location of features.

$$h_{i+1,x,y} = \max_{(j,k) \in N(x,y)} h_{i,j,k}$$

它通过降低空间分辨率来实现鲁棒性：通过“降低空间分辨率 (Reduce spatial resolution)”，网络可以“减少对位移和扭曲的敏感度 (Reduce sensitivity to shift and distortion)”。



In another word ...

池化的核心目标 (Objective of pooling) 是进行一次关键的转换：“将联合的特征表示转换成一个新的、更有用的表示，这个新的表示保留了重要信息，同时丢弃了不相关的细节 (preserves important information while discarding irrelevant detail)”。

1. In general terms, the objective of pooling is to transform the joint feature representation into a new, more usable one that *preserves important information while discarding irrelevant detail*, the crux of the matter being to determine what falls in which category
2. Achieving invariance to changes in position or lighting conditions, robustness to clutter, and compactness of representation, are all common goals of pooling

池化的共同目标 (Common goals of pooling) 包括实现对“位置...变化的不变性 (invariance to changes in position...)”，获得“表示的紧凑性 (compactness of representation)”。

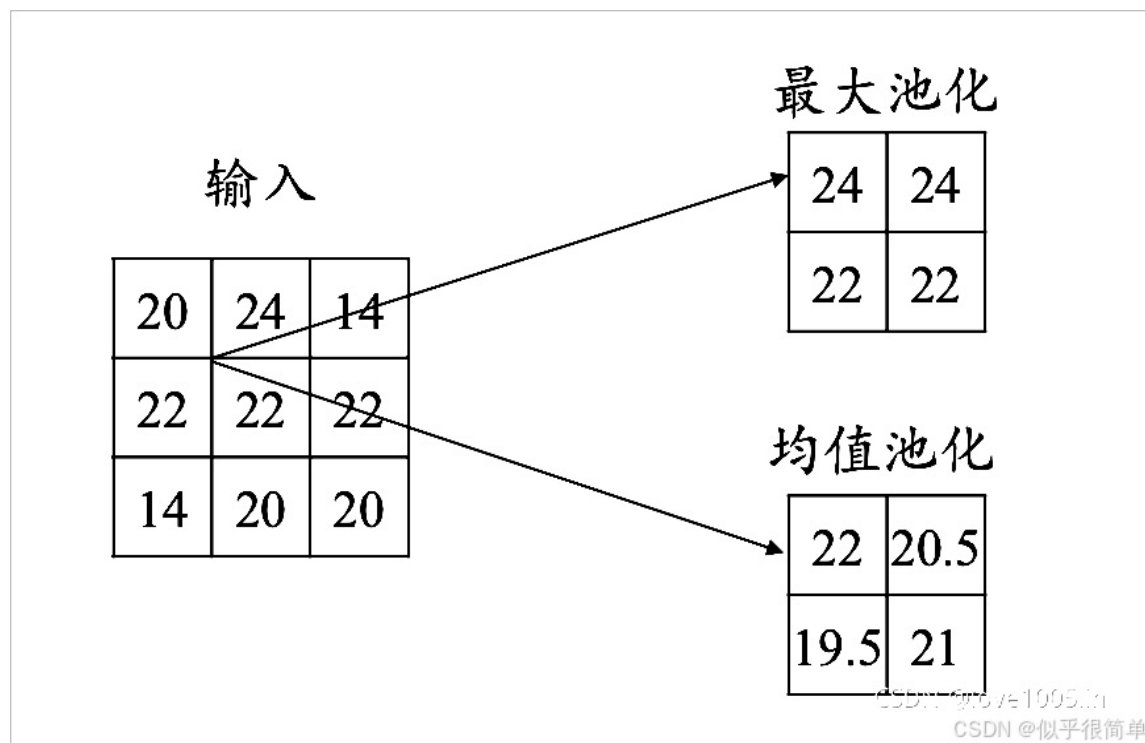
Max Pooling

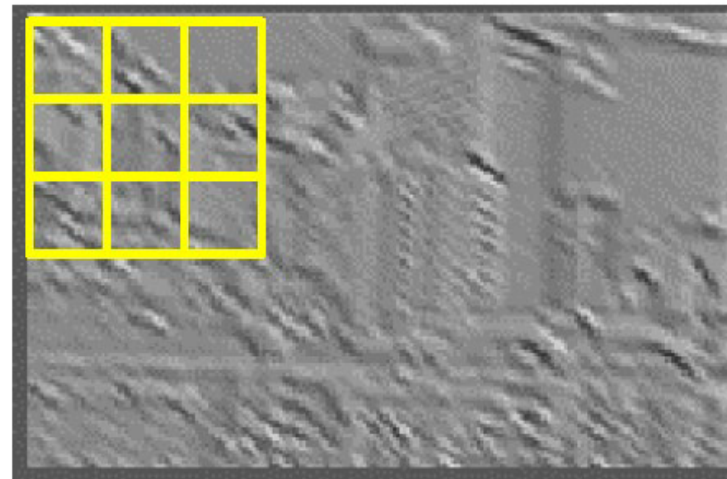
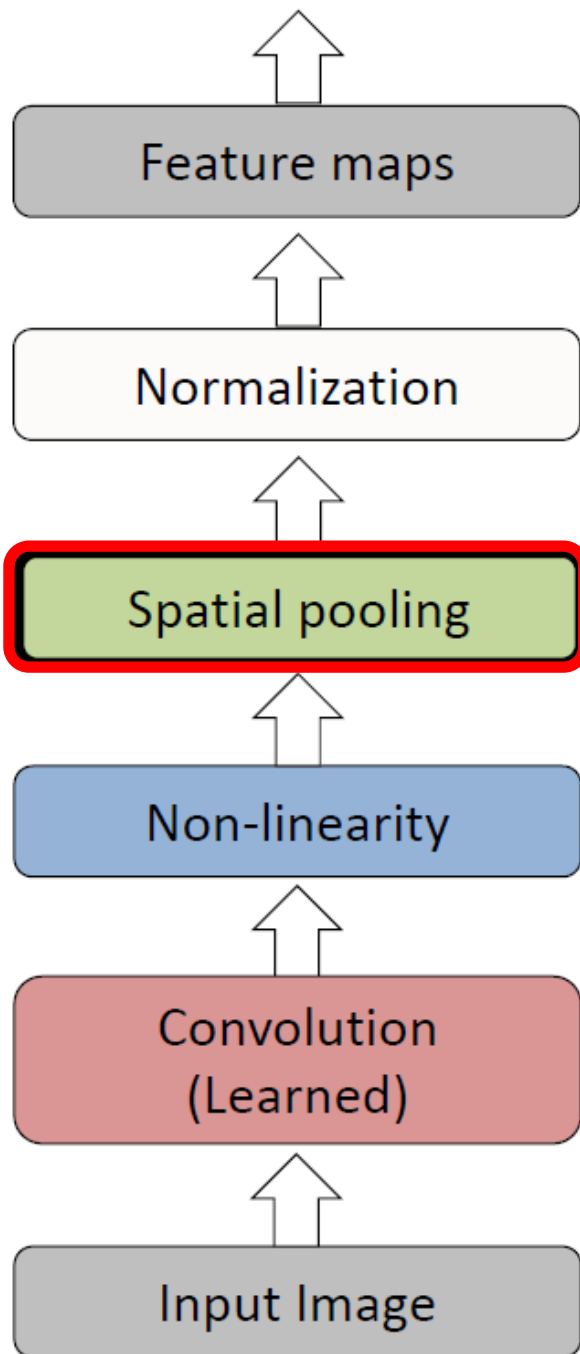
- *Max pooling* is a form of non-linear down-sampling, which partitions the input image into a set of non-overlapping rectangles and, for each such sub-region, outputs the maximum value
- Max pooling is useful in vision for two reasons
 - By eliminating non-maximal values, it reduces computation for upper layers
 - It provides a form of translation invariance
- Since it provides additional robustness to position, max-pooling is a “smart” way of reducing the dimensionality of intermediate representations

池化层 (Pooling Layer)

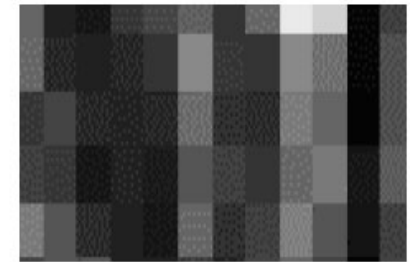
最大池化 (Max Pooling)：取局部区域的最大值（保留最显著特征，如边缘强度），例：2×2 最大池化（步长 = 2）将特征图尺寸减半。

平均池化 (Average Pooling)：取局部区域的平均值（保留整体趋势，常用于网络末尾）





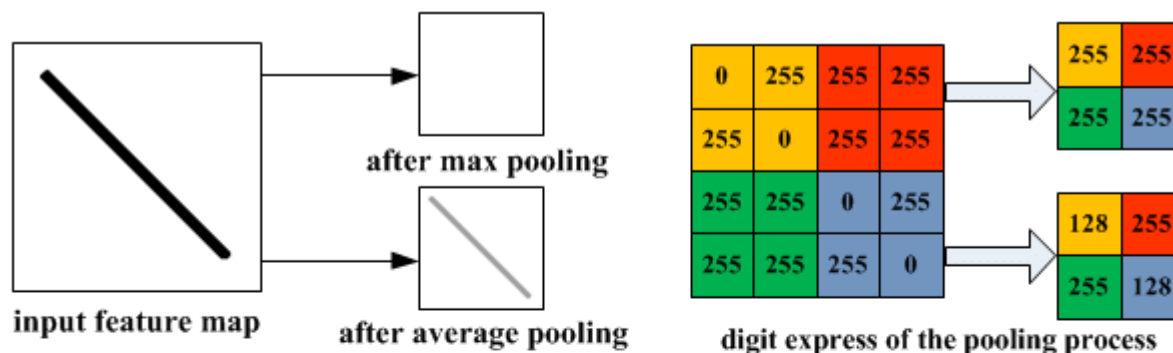
Max pooling



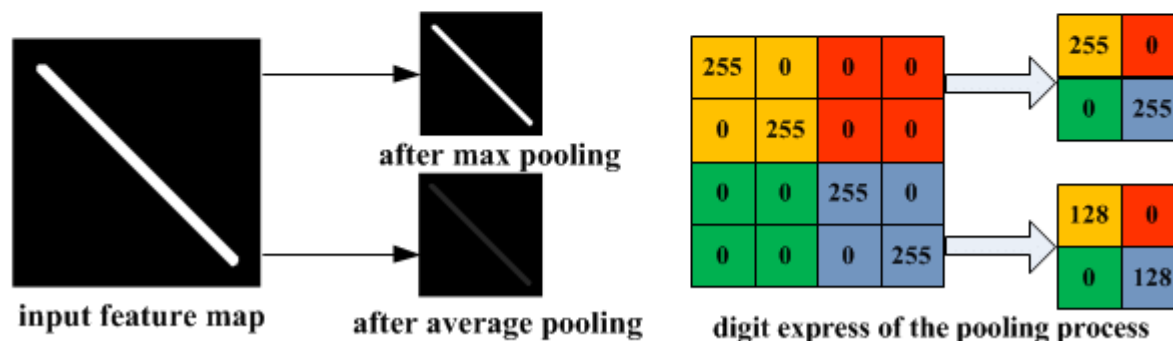
Max pooling:

- a non-linear down-sampling
- Provide *translation invariance*

Max Pooling & Average Pooling



(a) Illustration of max pooling drawback



(b) Illustration of average pooling drawback

Example

origin image



convolution 1



convolution 2

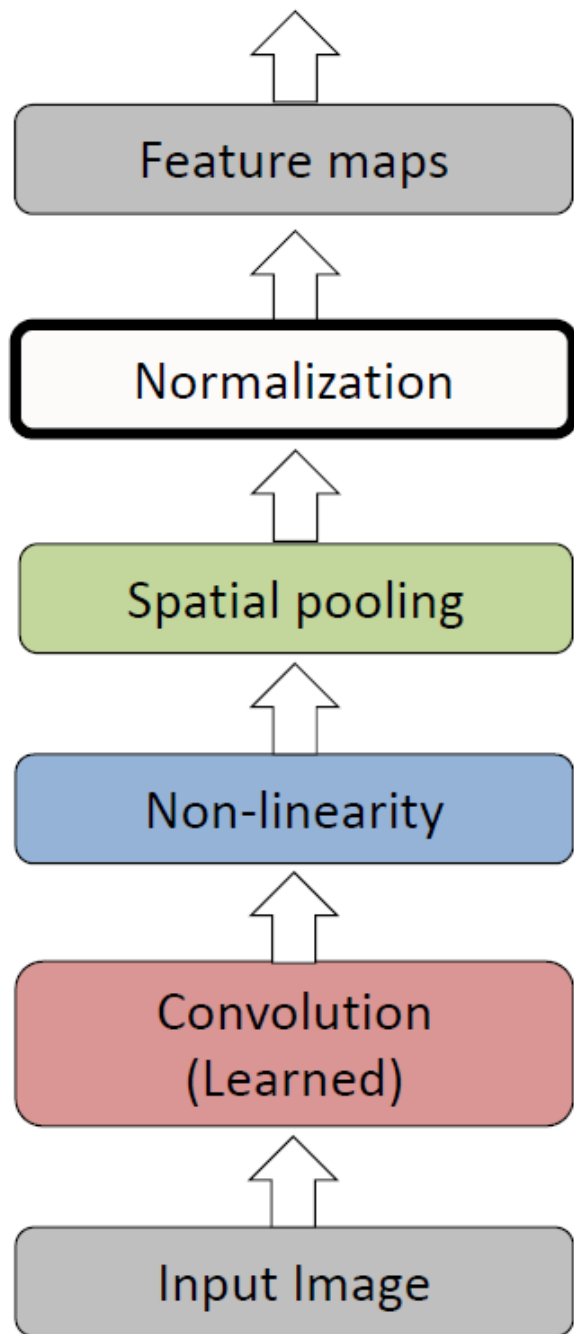


Down-sampled 1



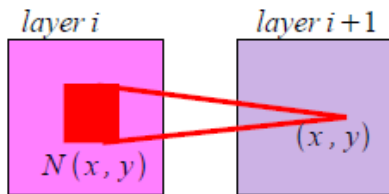
Down-sampled 2





归一化层

Local Contrast Normalization (over space / features)

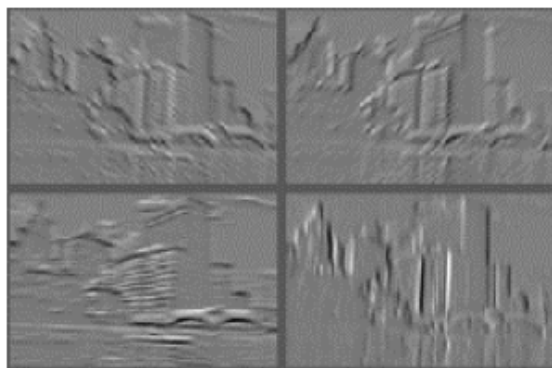


$$h_{i+1,x,y} = \frac{h_{i,x,y} - m_{i,x,y}}{\sigma_{i,x,y}}$$

功能：缓解“内部协变量偏移”（训练中每层输入分布不断变化），加速训练收敛，提升模型稳定性。

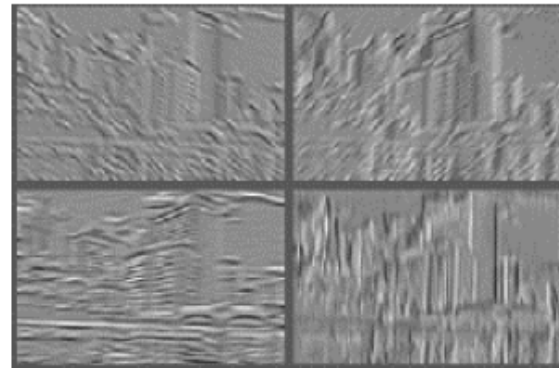
代表类型：

批量归一化（Batch Normalization, BN）：对每个批次的输入数据进行标准化（均值 = 0，方差 = 1），是目前主流的归一化方法，通常插入在卷积层与激活函数之间。



Feature Maps

通过减去一个局部区域的均值 m
然后除以该区域的标准差 σ
来重新计算神经元输出



Feature Maps

After Contrast
Normalization

What is really important ?

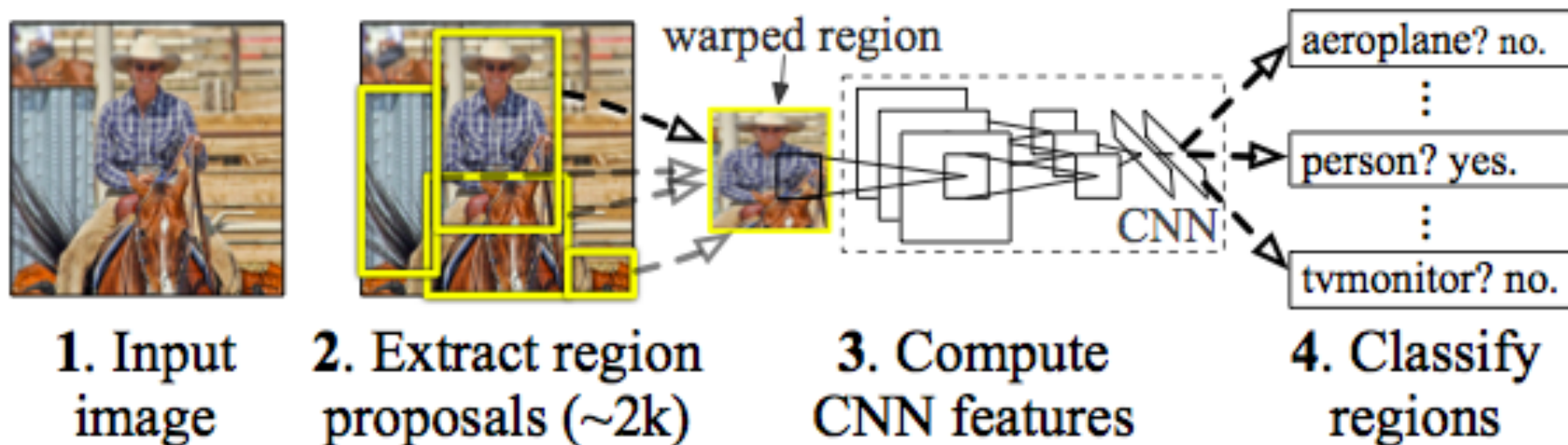
卷积层是最重要的部分

- The convolutional layers are the most important part
- A **pre-trained** network for image classification can be used for many different vision tasks.

一个在大数据集上训练好的CNN（比如在ImageNet上），其卷积层部分已经学会了如何从图片中提取各种有用的特征（从边缘到物体部件）。这个能力是通用的，我们可以把这部分“借”过来，用于我们自己的、全新的视觉任务，这就是所谓的“预训练网络 (pre-trained network)”或迁移学习。

Detection:

R-CNN: *Regions with CNN features*



GoogLeNet

- 22 layers' deep networks



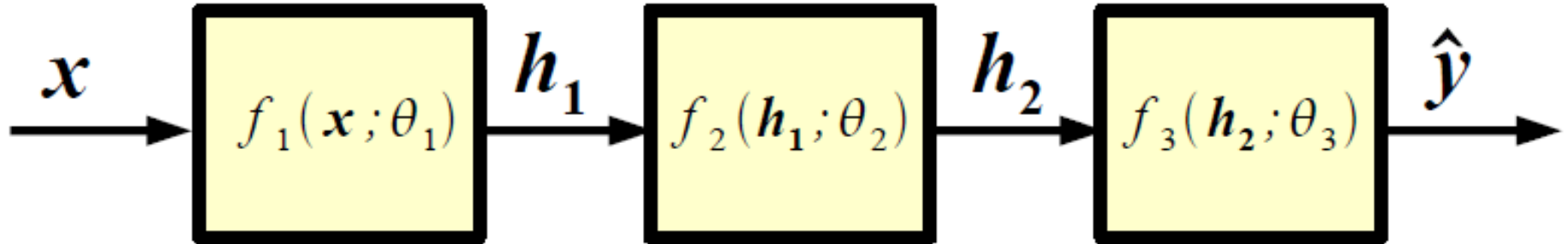
Convolution

Max Pooling

Softmax

Concatenation

Idea of CNN training

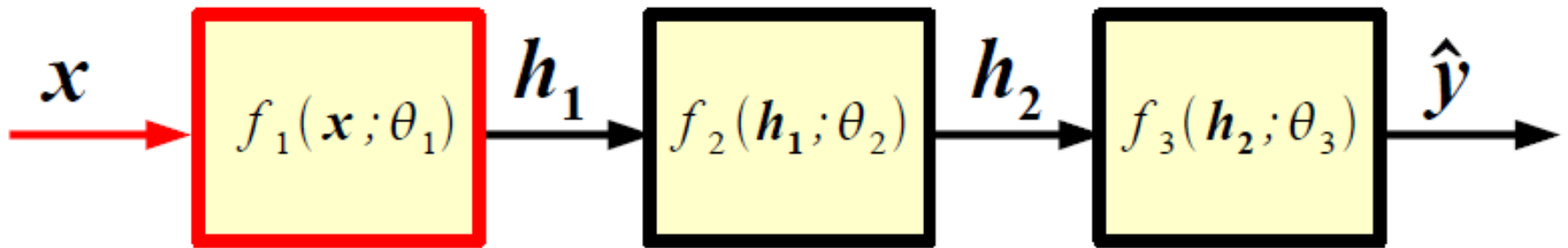


NOTE:

In practice, any differentiable non-linear transformation is potentially good.

每个函数 f 都有自己的一套参数 θ 。这些 θ 就代表了我们之前讨论的所有可学习的权重 (weights) 和偏置 (biases)。CNN的“训练”过程，本质上就是通过反向传播和梯度下降算法，不断调整这些 θ 参数，使得对于一个给定的输入 x （图片），网络的最终输出 $\hat{y}$ （预测结果）能够尽可能地接近真实标签 y 。

Forward Propagation (FP)



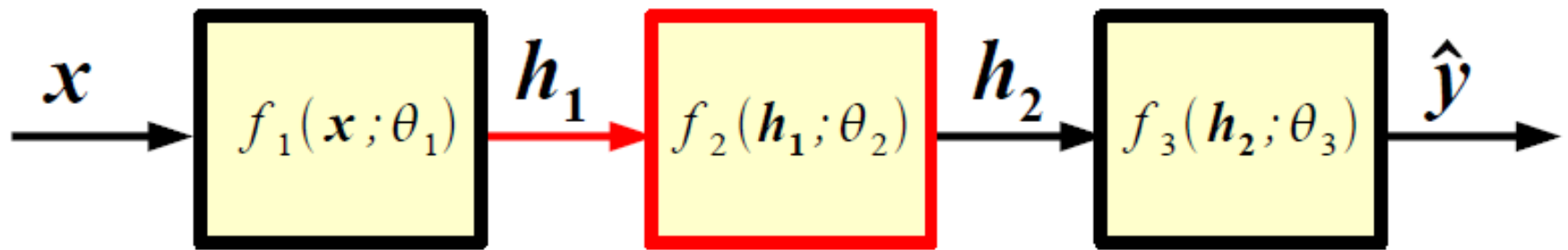
1) Given x compute: $h_1 = f_1(x; \theta_1)$

For instance,

$$h_1 = \max(0, W_1 x + b_1)$$

ReLU Rectified Linear Units.

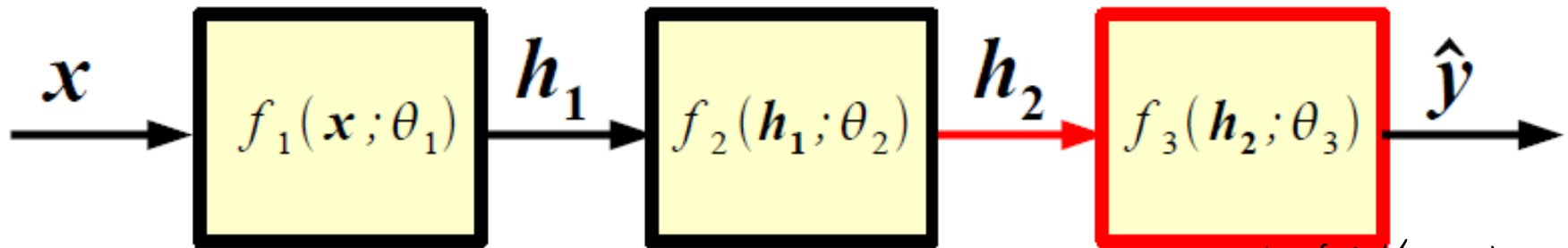
Forward Propagation (FP)



1) Given x compute: $h_1 = f_1(x; \theta_1)$

2) Given h_1 compute: $h_2 = f_2(h_1; \theta_2)$

Forward Propagation (FP)



1) Given x compute: $h_1 = f_1(x; \theta_1)$

2) Given h_1 compute: $h_2 = f_2(h_1; \theta_2)$

3) Given h_2 compute: $\hat{y} = f_3(h_2; \theta_3)$

分类任务中输出层采用 softmax
这个函数将最后的原始输出
logits 转换成一个概率分布
公式 $\hat{y} = p(\text{class} = i | x)$
输出向量中第 i 个值, 代表了
输入 x 属于第 i 类的概率

For instance,

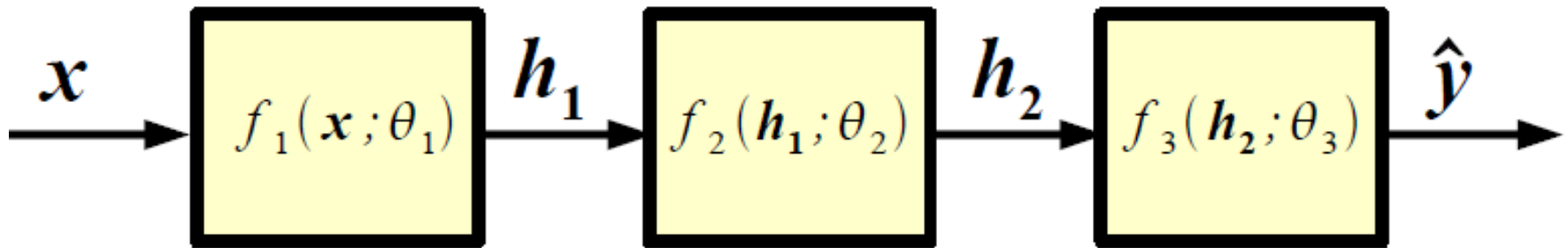
$$\hat{y}_i = p(\text{class} = i | x) = \frac{e^{W_{3i} h_2 + b_{3i}}}{\sum_k e^{W_{3k} h_2 + b_{3k}}}$$

Softmax output



probability that input x belongs
to one of the predefined classes

Forward Propagation (FP)



- 1) Given x compute: $h_1 = f_1(x; \theta_1)$
- 2) Given h_1 compute: $h_2 = f_2(h_1; \theta_2)$
- 3) Given h_2 compute: $\hat{y} = f_3(h_2; \theta_3)$

This is the typical processing at test time.

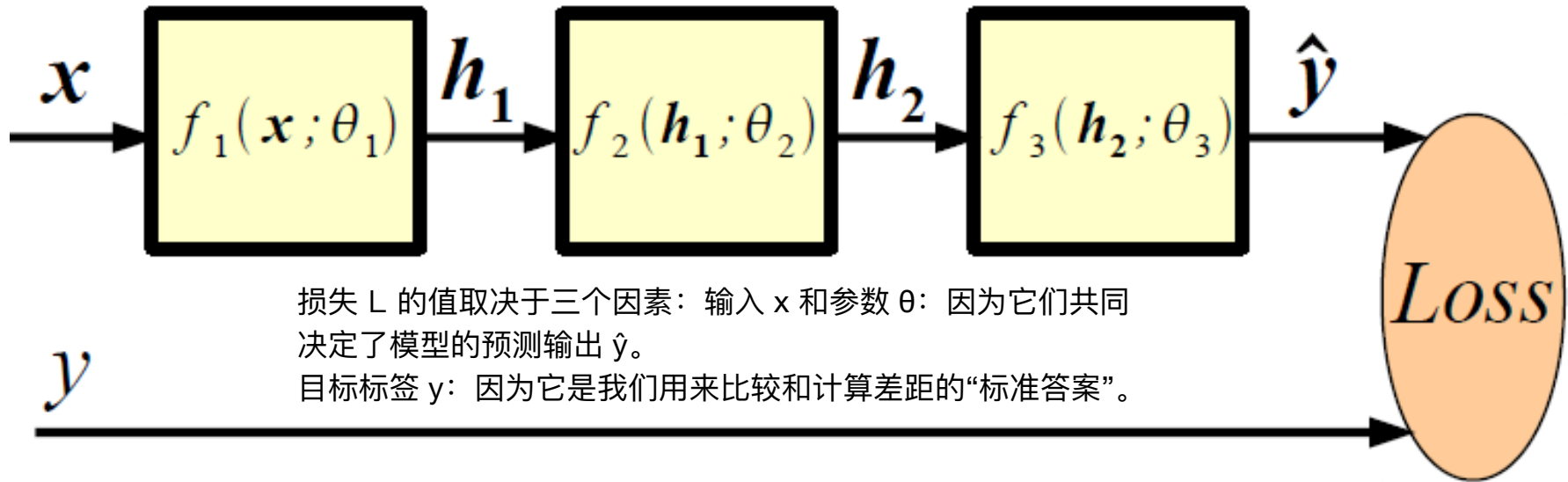
At training time, we need to compute an **error measure** and tune the parameters to decrease the error.

我们刚刚描述的整个前向传播过程，是网络在测试（或预测）时的典型处理流程。我们输入一张未见过的图片，网络通过Forward Propagation 给出一个预测结果。

但在训练时，这仅仅是第一步。在得到预测 $\hat{y}$ 之后，我们还需要“计算一个误差度量 (error measure)”（即将 $\hat{y}$ 与真实标签 y 进行比较），然后“调整参数来降低这个误差”。这个调整参数的过程，就是我们尚未讨论的反向传播 (Backpropagation)。

Loss Function

损失函数 $L(x, y; \theta)$ 是一个用来衡量“模型在训练集上拟合得有多好 (how well the model fits the training set)”的度量。简单来说，它量化了模型的预测 $\hat{y}$ 与真实标签 y 之间的差距。如果差距大，损失就大；如果预测很准，损失就很小。



损失 L 的值取决于三个因素：输入 x 和参数 θ ：因为它们共同决定了模型的预测输出 $\hat{y}$ 。

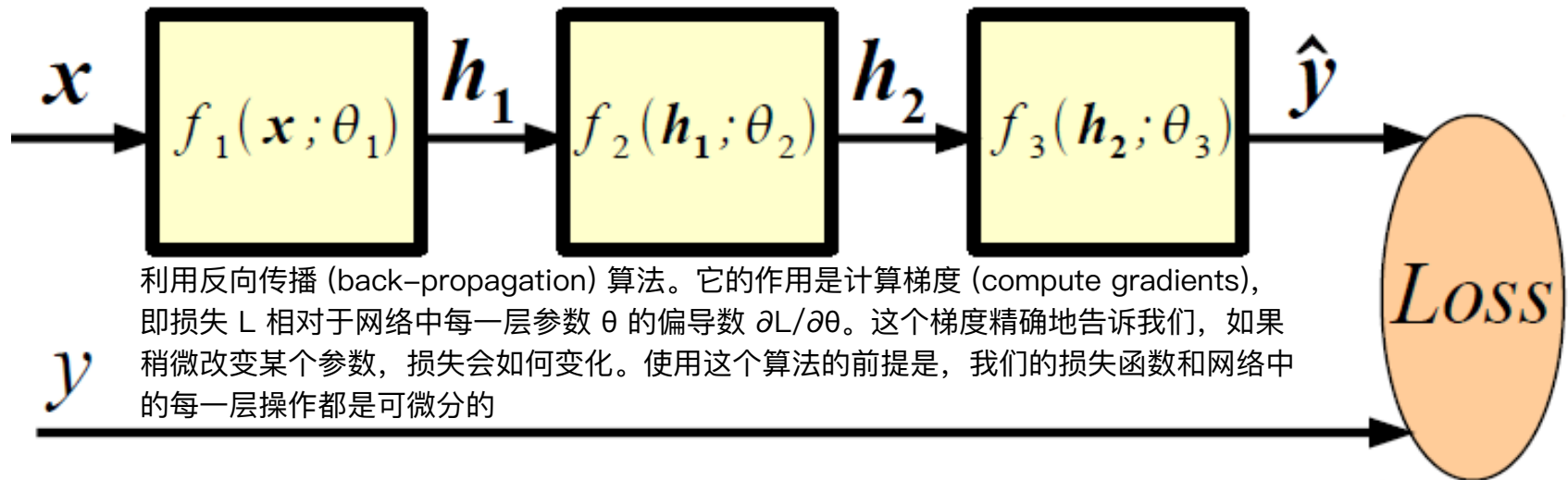
目标标签 y ：因为它是我们用来比较和计算差距的“标准答案”。

The measure of how well the model fits the training set can be given by a suitable **loss function**: $L(x, y; \theta)$

The loss depends on the input x , the target label y , and the parameters θ

训练的核心目标：调整参数 θ （也就是网络中所有的权重和偏置），使得在整个训练数据集上，损失函数 L 的总值尽可能小。

Backward Propagation (BP)



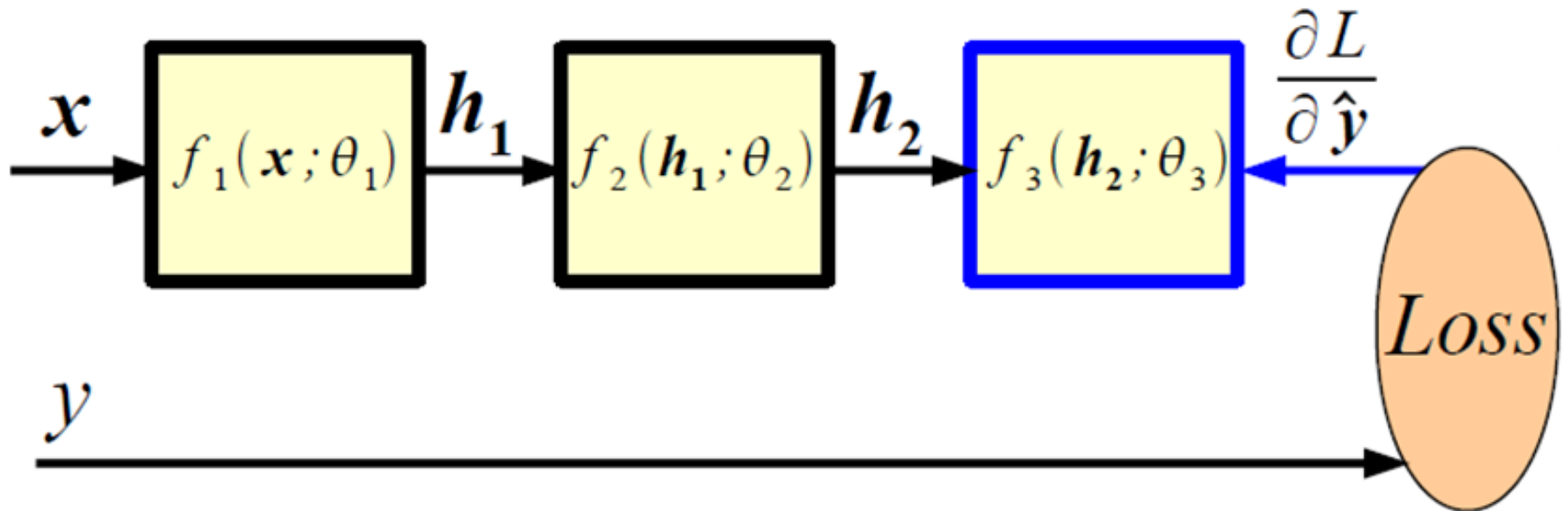
如何调整参数来降低损失?

Q.: how to tune the parameters to decrease the loss?

If loss is differentiable we can compute gradients.
We can use **back-propagation**, to compute the gradients w.r.t. parameters at the lower layers

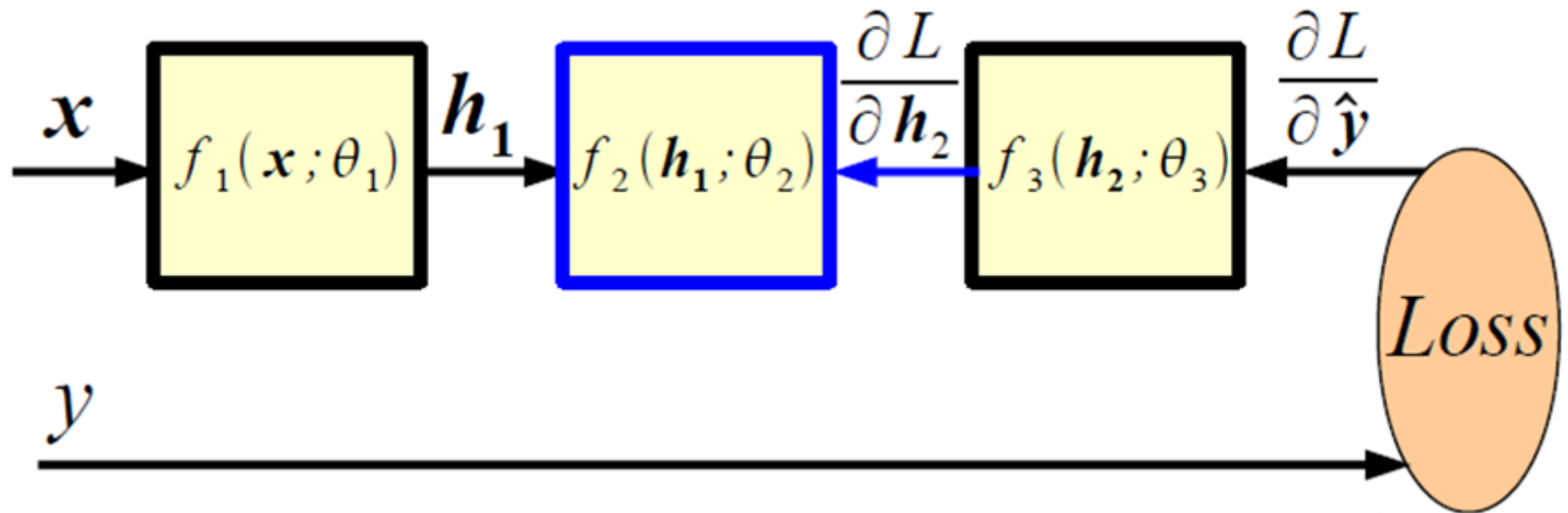
Backward Propagation (BP)

起点是损失函数



我们计算损失 L 相对于网络最终输出 $\hat{y}$ 的梯度 $\partial L / \partial \hat{y}$ 。这代表了“初始的误差信号”。

Backward Propagation (BP)



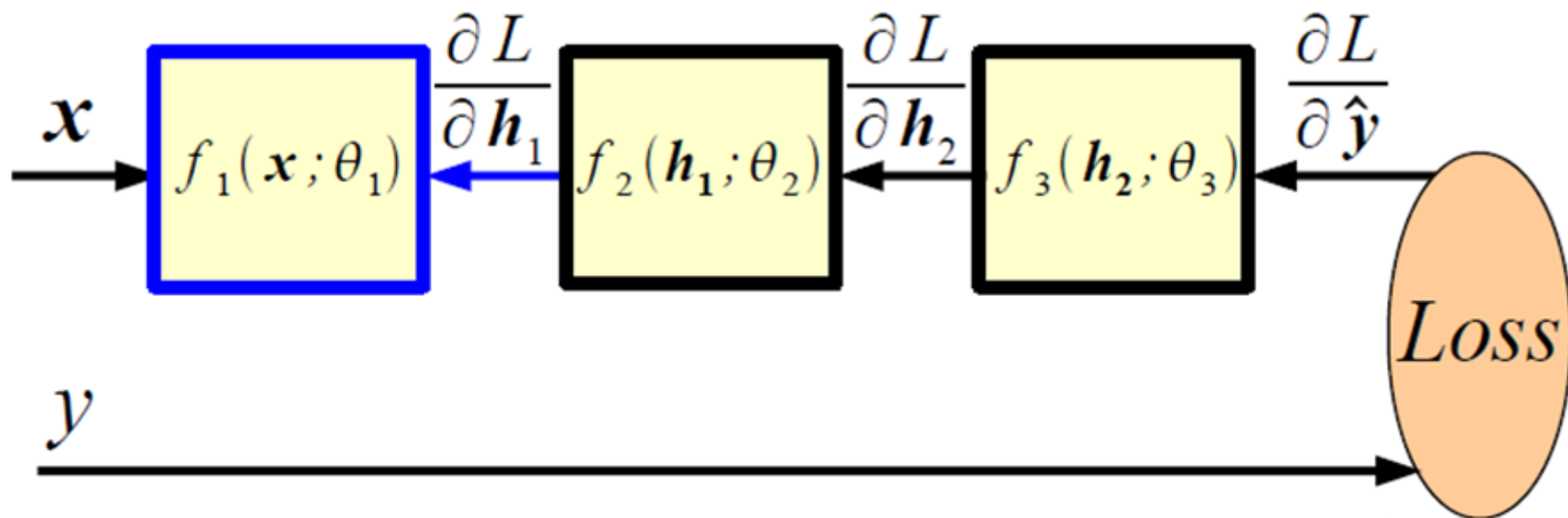
逐层向后传递: 这个误差信号会从网络的末端开始, 逐层向前 (反向) 传递。

误差信号首先通过 f_3 , 计算出损失 L 相对于 h_2 的梯度 $\partial L / \partial h_2$ 。

接着, 这个新的梯度信号又通过 f_2 , 计算出损失 L 相对于 h_1 的梯度 $\partial L / \partial h_1$ 。

这个过程会一直持续到网络的最前端。

Backward Propagation (BP)



虽然图中主要展示的是损失相对于各层输出 h 的梯度，但反向传播的最终目的是利用这些中间梯度，通过链式法则计算出损失 L 相对于每一层参数 θ 的梯度（即 $\partial L / \partial \theta_1$, $\partial L / \partial \theta_2$, $\partial L / \partial \theta_3$ ）。一旦我们知道了这些梯度，我们就知道了调整每一个参数以降低损失的“正确方向”。

Optimization

随机梯度下降

这是我们之前 FP -> Loss -> BP -> Update 循环中，Update 这一步的具体实现算法。

Stochastic Gradient Descent (on mini-batches):

$$\theta \leftarrow \theta - \eta \frac{\partial L}{\partial \theta}, \eta \in R$$

θ : 代表网络中需要优化的任意一个参数（权重或偏置）。
 $\partial L / \partial \theta$: 这是我们通过反向传播计算出的，损失函数 L 相对于参数 θ 的梯度。它指明了能让损失函数增长最快的方向。
 η (eta): 这是一个名为学习率 (learning rate) 的超参数。它是一个很小的正数，用来控制每次参数更新的步长大小。

核心思想: 整个公式的含义是，为了让损失 L 减小，我们应该将参数 θ 沿着梯度 $\partial L / \partial \theta$ 的相反方向（由负号 - 表示）更新一小步（步长由 η 控制）。这个过程就像一个盲人下山，每走一步，他都会用脚探测一下哪个方向是下坡最陡峭的，然后朝那个方向挪动一小步，如此反复，直到走到山谷（损失函数的最小值点）。

带有动量的随机梯度下降 (对基础 SGD 改进)

Stochastic Gradient Descent with Momentum:

$$\theta \leftarrow \theta - \eta \Delta$$

$$\Delta \leftarrow 0.9 \Delta + \frac{\partial L}{\partial \theta}$$

我们不再直接使用当前计算出的梯度 $\partial L / \partial \theta$ 来更新参数。取而代之，我们维护一个“动量”向量 Δ 。在每一步，这个 Δ 都会累加上当前梯度的一部分，同时保留一小部分 (0.9) 旧的动量。这本质上是在计算梯度的一个指数加权移动平均值。最终，我们用这个更平滑、积累了历史梯度信息的动量向量 Δ 来更新参数 θ 。

核心思想: 这就像一个从山上滚下来的重球。它不仅会沿着当前最陡峭的方向滚，还会因为自身的**惯性（动量）**继续向前冲。这有助于小球冲过一些小的颠簸（局部最小值），并且在梯度方向一致的“大直道”上加速前进，从而更快、更稳定地到达山底。

Transfer Learning (TL)

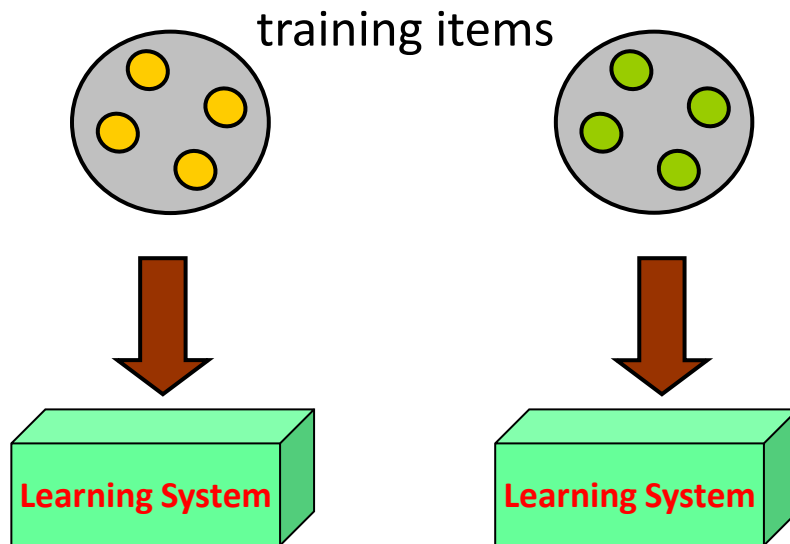
我们可以拿一个已经在大规模数据集（例如，包含数百万张图片的ImageNet）上训练好的CNN模型，然后将其强大的图像识别“知识”——也就是它在卷积层中学到的通用特征提取能力——应用到一个全新的、可能只有少量数据的图像识别任务上。这直接关联到你之前的“预训练网络 (pre-trained network)”的概念。

- The ability of a system to recognize and apply knowledge and skills learned in previous tasks to novel tasks (in new domains)
- TL is motivated by human learning: people can often transfer knowledge learnt previously to novel situations
 - Chinese → English
 - mathematics → computer science
 - network technology for internet → social network

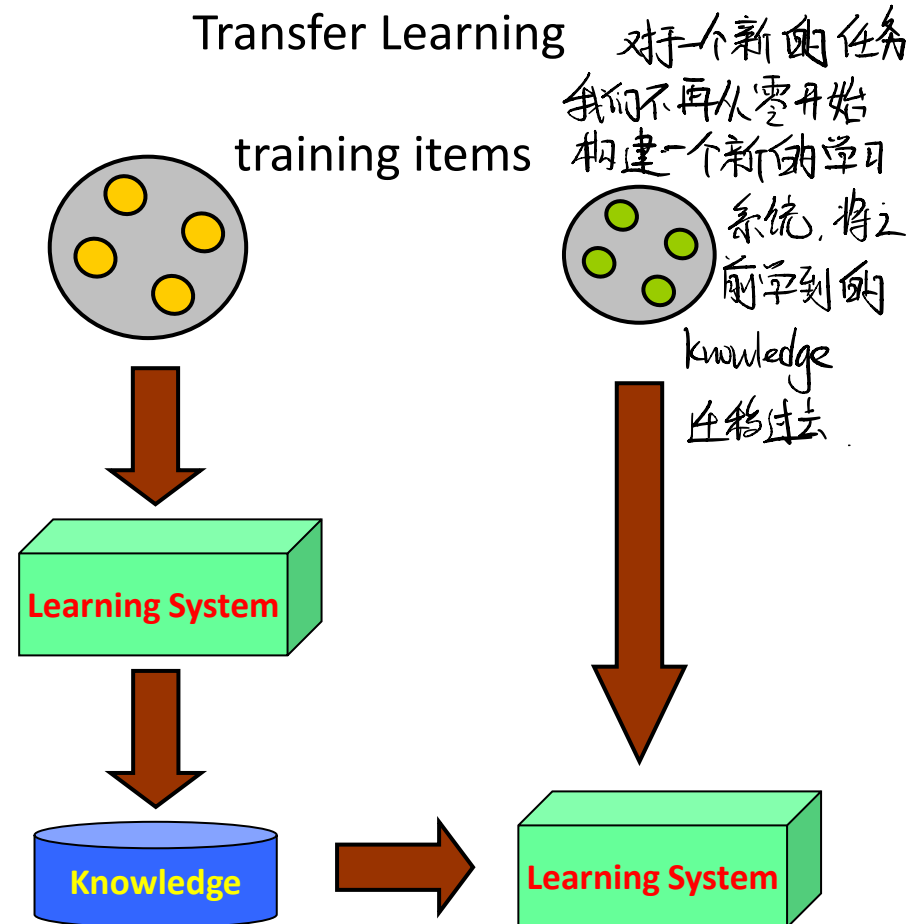
一个CNN在识别猫、狗、汽车等上千种物体时学到的如何识别“边缘”、“纹理”、“形状”、“眼睛”、“轮子”等视觉特征的“技能”，同样可以被用来识别医学影像中的细胞或识别不同种类的花朵。

Traditional ML vs. TL

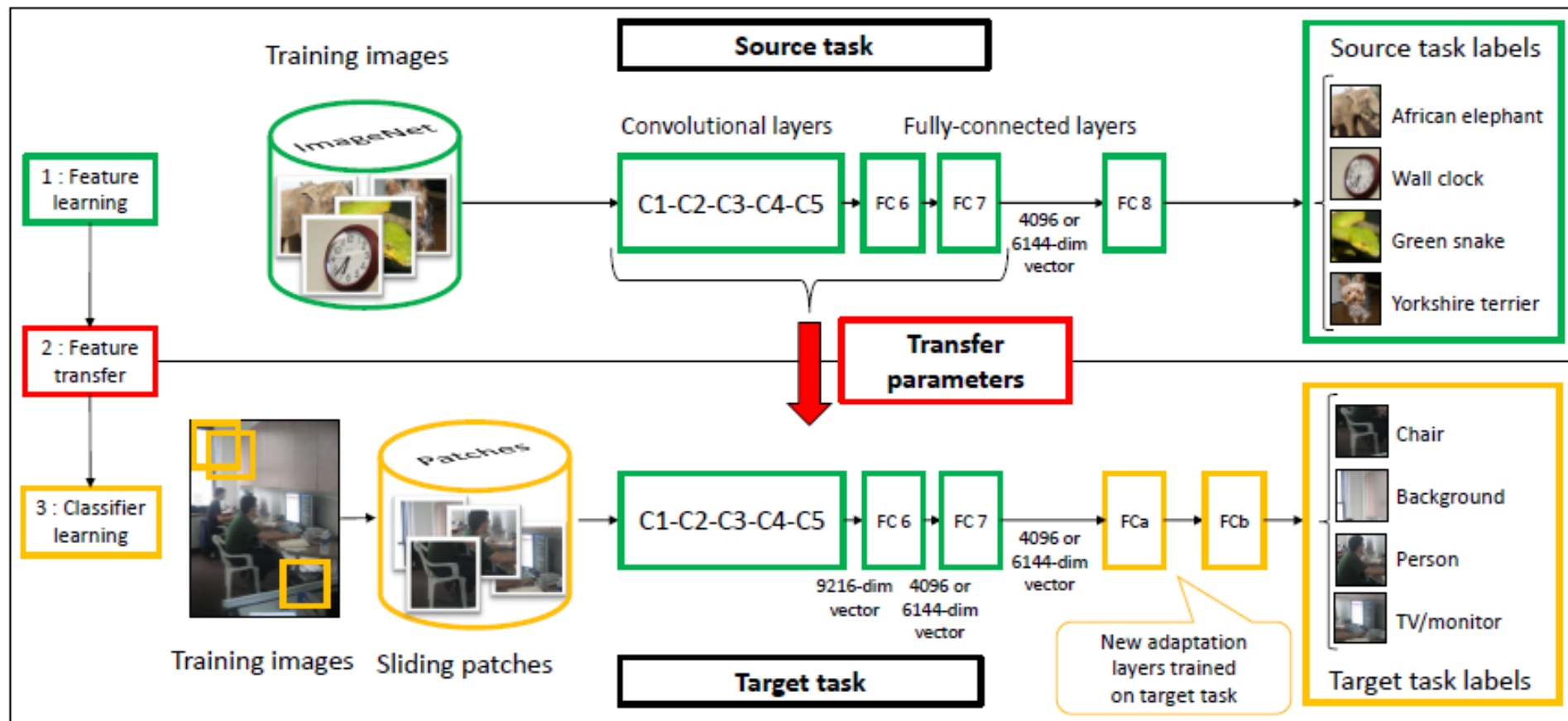
Learning Process of
Traditional ML



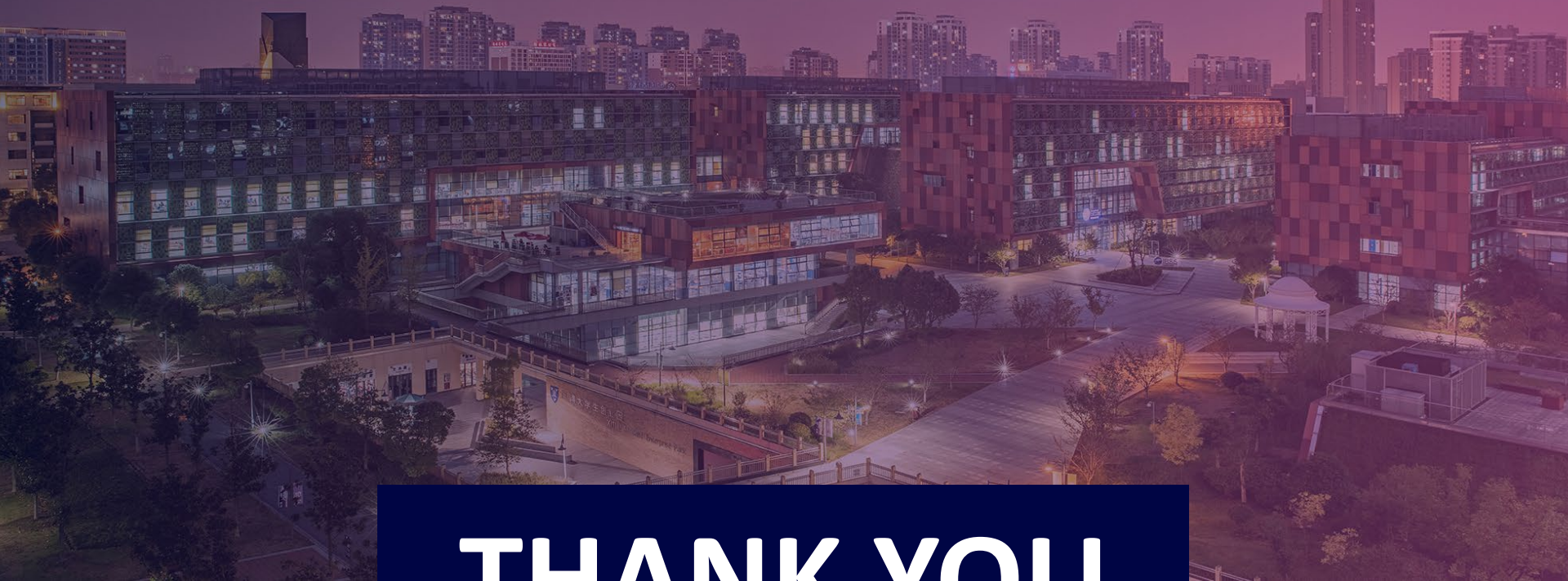
Learning Process of
Transfer Learning



Deep CNN for Knowledge Transfer



The network is trained on the source task (top row) with a large amount of available labelled images. Pre-trained parameters of the internal layers of the network (C1-FC7) are then transferred to the target tasks (bottom row). To compensate for the different image statistics (type of objects, typical viewpoints, imaging conditions) of the source and target data, an adaptation layer (fully connected layers FCa and FCb) is added and trained on the labelled data of the target task.



THANK YOU



VISIT US

WWW.XJTLU.EDU.CN



FOLLOW US

@XJTLU



Xi'an Jiaotong-Liverpool University

西交利物浦大學